

CAMS

Computer Account Management System

WRITTEN USE CASES

219 use cases across 29 modules and 3 actors

Administrator 91 · Teacher 107 · Student 21

About this document

Each written use case below follows the ten fields set out for a use case specification: use case name, purpose, actors, input parameters, output parameters, pre-condition, post-condition, successful scenario, exception scenario and additional remarks. Every module is drawn first, with its figure caption, and the use cases inside it follow.

Each use case is named after the function in the CAMS source that implements it. A use case named `CREATE STUDENT` describes `AdminController.CreateStudent`; one named `LOGIN ASYNC` describes `AuthenticationService.LoginAsync`, used where the behaviour has no endpoint of its own and is reached through an include relationship.

The fields that can be got wrong are read out of the source rather than written from memory. Input parameters come from the action signature and its `[Bind]` list, so a use case cannot name a field the form does not submit. The pre-condition comes from the `[Authorize]` attribute, including the `[TeacherSharedAction]` marker that admits a teacher to an administrator action. The antiforgery step appears only where the controller validates one, and a hub method is described as a SignalR relay rather than a form post, because that is what it is.

The companion diagram is `CAMS-Use-Case-Diagram.drawio`, in the same folder.

ADMIN

Process Log In AccountController

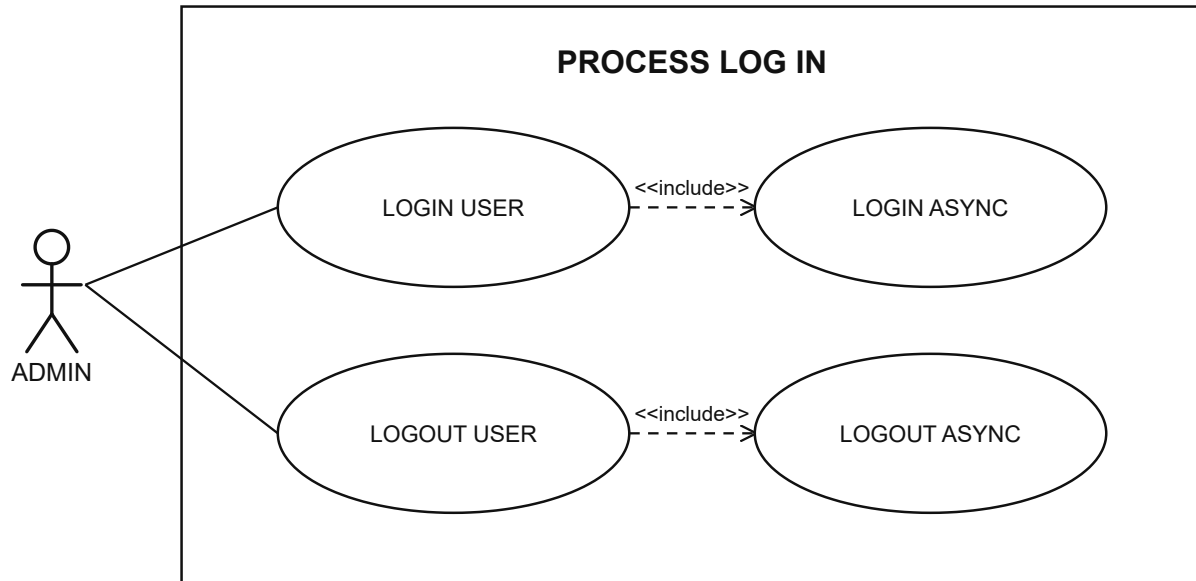


Figure 3.1: System Use Case for Process Log In

Use Case Name: LOGIN USER

Purpose: Let a person sign in to the CAMS web portal with a username and password, and place them in the part of the system their role allows.

Actors:

- Admin (Primary Actor)

Input Parameters:

- `username : string`
- `password : string`

Output Parameters: An authentication cookie carrying the account role as a claim, and a redirect to the landing page for that role.

Pre-Condition: None. This is the sign-in endpoint and is reachable without an account session; the CAMS server must be running and reachable.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the CAMS sign-in page.
2. The Admin enters a username and a password and submits the form.
3. The server validates the antiforgery token that accompanied the form.
4. The server checks the credentials against each account table in turn and finds the matching account.
5. The server confirms the account is active and not locked out.
6. The server issues an authentication cookie carrying the role as a claim.
7. The Admin is redirected to the landing page for that role.

Exception Scenario:

- **The username matches no account** — the page reports that the sign-in failed, without saying which half was wrong.
- **The password does not match the stored hash** — the failed-attempt counter is raised and the same message is shown.
- **The account is locked out** — the sign-in is refused until the lockout expires, even with the right password.
- **The account is deactivated** — the sign-in is refused and the person is told to contact an administrator.

Additional Remarks:

- Implemented by `AccountController.Login` (POST).
- Appears in the *PROCESS LOG IN* module of the use case diagram.
- Pulls in **LOGIN ASYNC** (<<include>>).

Use Case Name: LOGOUT USER

Purpose: End the signed-in session and clear the authentication cookie, so the next visitor to the browser starts as an anonymous user.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The authentication cookie is cleared and the browser is returned to the sign-in page.

Pre-Condition: The caller is signed in.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin chooses to sign out.
2. The server closes any lab session the account still has open.
3. The server clears the authentication cookie.
4. The browser is returned to the sign-in page as an anonymous visitor.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AccountController.Logout` (POST).
- Appears in the *PROCESS LOG IN* module of the use case diagram.

- Pulls in **LOGOUT ASYNC** (<<include>>).

Use Case Name: LOGIN ASYNC

Purpose: Check the supplied password against the stored hash, confirm the account is active and not locked out, and decide which of the three account tables the person belongs to.

Actors:

- Admin (Primary Actor)
- None; this behaviour runs inside **LOGIN USER** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: A `LoginResult` naming the role, the account id and the display name, or a result saying the credentials were rejected.

Pre-Condition:

- The sign-in use case has supplied a username and a password.
- **LOGIN USER** has reached the point where this is always performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The sign-in use case passes the username, the password, the workstation name and the caller address.
2. The service looks the username up in the administrator, teacher and student tables.
3. The stored hash is verified against the supplied password.
4. The active flag and the lockout expiry are checked.
5. On success the failed-attempt counter is cleared and the role and account id are returned; on failure the counter is raised.

Exception Scenario:

- The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.

Additional Remarks:

- Implemented by `LoginAsync`.
- Appears in the *PROCESS LOG IN* module of the use case diagram.
- Drawn as `<<include>>` from **LOGIN USER**.

Use Case Name: LOGOUT ASYNC

Purpose: Close any lab session the student still has open and record the sign-out.

Actors:

- Admin (Primary Actor)
- None; this behaviour runs inside **LOGOUT USER** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The behaviour completes and its effect is visible to the use case that includes it.

Pre-Condition:

- The caller is signed in.
- **LOGOUT USER** has reached the point where this is always performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The including use case reaches the point where this behaviour is required.
2. The server runs `LogoutAsync` and applies its result.
3. Control returns to the including use case, which continues.

Exception Scenario:

- The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.

Additional Remarks:

- Implemented by `LogoutAsync`.
- Appears in the *PROCESS LOG IN* module of the use case diagram.
- Drawn as <<include>> from **LOGOUT USER**.

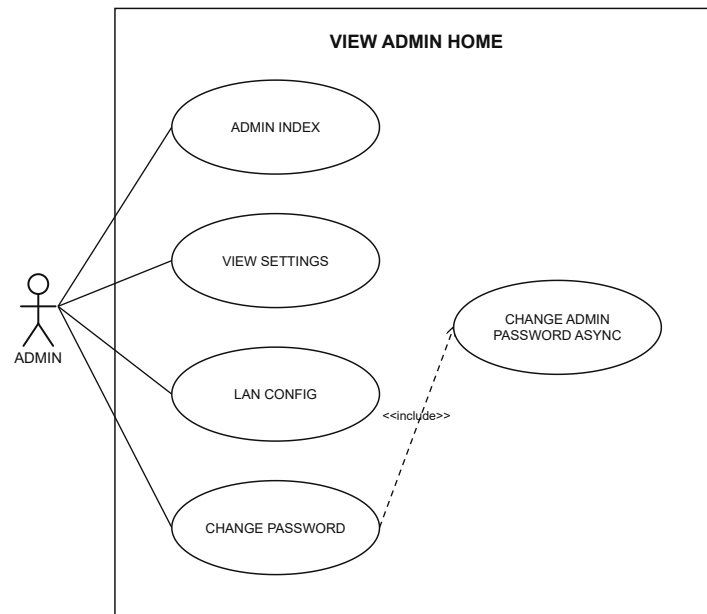
View Admin Home AdminController

Figure 3.2: System Use Case for View Admin Home

Use Case Name: ADMIN INDEX

Purpose: Show the administrator landing page, summarising the state of the laboratory.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.Index` (GET).
- Appears in the *VIEW ADMIN HOME* module of the use case diagram.

Use Case Name: VIEW SETTINGS

Purpose: Show the settings page for the signed-in user.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as an administrator or a teacher.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.Settings (GET)`.
- Appears in the *VIEW ADMIN HOME* module of the use case diagram.

Use Case Name: LAN CONFIG

Purpose: Show the network configuration the server detected, as a read-only diagnostic.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as an administrator or a teacher.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.LanConfig (GET)`.
- Appears in the *VIEW ADMIN HOME* module of the use case diagram.

Use Case Name: CHANGE PASSWORD

Purpose: Let the signed-in user replace their own password after proving they know the current one.

Actors:

- Admin (Primary Actor)

Input Parameters: `input : PasswordChangeInput`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as an administrator or a teacher.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.

- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.ChangePassword` (POST).
- Appears in the *VIEW ADMIN HOME* module of the use case diagram.
- Pulls in **CHANGE ADMIN PASSWORD ASYNC** (<<include>>).

Use Case Name: CHANGE ADMIN PASSWORD ASYNC

Purpose: Verify the current password and replace the stored hash for an administrator account.

Actors:

- Admin (Primary Actor)
- None; this behaviour runs inside **CHANGE PASSWORD** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The behaviour completes and its effect is visible to the use case that includes it.

Pre-Condition:

- The caller is signed in.
- **CHANGE PASSWORD** has reached the point where this is always performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The including use case reaches the point where this behaviour is required.
2. The server runs `ChangeAdminPasswordAsync` and applies its result.
3. Control returns to the including use case, which continues.

Exception Scenario:

- The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.

Additional Remarks:

- Implemented by `ChangeAdminPasswordAsync`.
- Appears in the *VIEW ADMIN HOME* module of the use case diagram.
- Drawn as <<include>> from **CHANGE PASSWORD**.

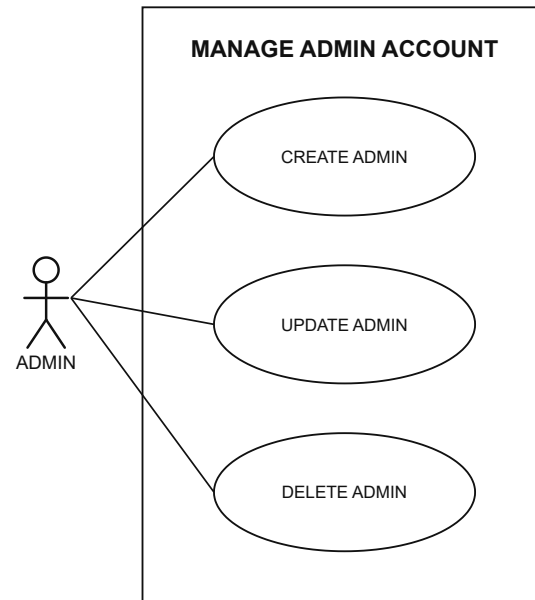
Manage Admin Account AdminController

Figure 3.3: System Use Case for Manage Admin Account

Use Case Name: CREATE ADMIN

Purpose: Record a new administrator account in the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `admin : Admin`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as an administrator or a teacher.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.CreateAdmin` (POST).
- Appears in the *MANAGE ADMIN ACCOUNT* module of the use case diagram.

Use Case Name: UPDATE ADMIN

Purpose: Amend the stored details of an existing administrator account.

Actors:

- Admin (Primary Actor)

Input Parameters:

- `admin : Admin`
- `newPassword : string?`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as an administrator or a teacher.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.

- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.UpdateAdmin` (POST).
- Appears in the *MANAGE ADMIN ACCOUNT* module of the use case diagram.

Use Case Name: DELETE ADMIN

Purpose: Remove a administrator account from the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.

- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `AdminController.DeleteAdmin` (POST).
- Appears in the *MANAGE ADMIN ACCOUNT* module of the use case diagram.

Manage Teacher Account `AdminController`

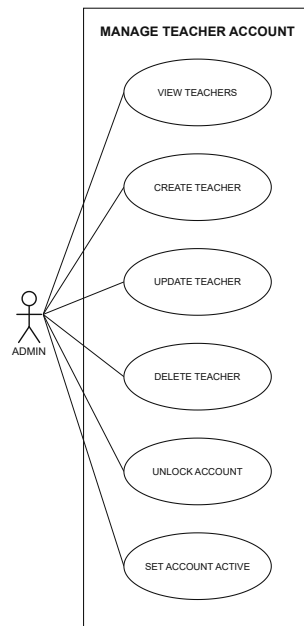


Figure 3.4: System Use Case for Manage Teacher Account

Use Case Name: VIEW TEACHERS

Purpose: List the teachers the signed-in user is allowed to see.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.Teachers (GET)`.
- Appears in the *MANAGE TEACHER ACCOUNT* module of the use case diagram.

Use Case Name: CREATE TEACHER

Purpose: Record a new teacher account in the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `teacher : Teacher`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.CreateTeacher` (POST).
- Appears in the *MANAGE TEACHER ACCOUNT* module of the use case diagram.

Use Case Name: UPDATE TEACHER

Purpose: Amend the stored details of an existing teacher account.

Actors:

- Admin (Primary Actor)

Input Parameters:

- `teacher : Teacher`
- `newPassword : string?`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.UpdateTeacher` (POST).
- Appears in the *MANAGE TEACHER ACCOUNT* module of the use case diagram.

Use Case Name: DELETE TEACHER

Purpose: Remove a teacher account from the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `AdminController.DeleteTeacher` (POST).
- Appears in the *MANAGE TEACHER ACCOUNT* module of the use case diagram.

Use Case Name: UNLOCK ACCOUNT

Purpose: Clear the lockout on an account that has been locked by repeated failed sign-in attempts.

Actors:

- Admin (Primary Actor)

Input Parameters:

- `accountRole : AccountRole`
- `id : int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.UnlockAccount` (POST).
- Appears in the *MANAGE TEACHER ACCOUNT* module of the use case diagram.

Use Case Name: SET ACCOUNT ACTIVE

Purpose: Activate or deactivate an account without deleting it, so a person can be kept out of the system while their records survive.

Actors:

- Admin (Primary Actor)

Input Parameters:

- `accountRole : AccountRole`
- `id : int`
- `isActive : bool`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.

6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.SetAccountActive (POST)`.
- Appears in the *MANAGE TEACHER ACCOUNT* module of the use case diagram.

Manage Student Account `AdminController`

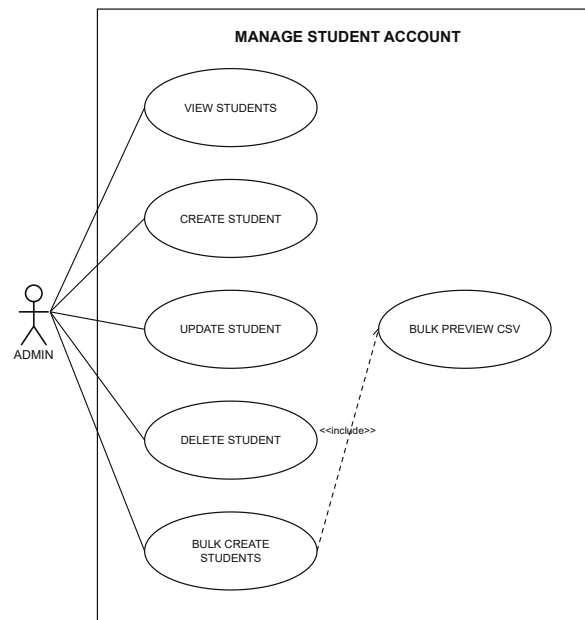


Figure 3.5: System Use Case for Manage Student Account

Use Case Name: VIEW STUDENTS

Purpose: List the students the signed-in user is allowed to see.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.Students (GET)`.
- Appears in the *MANAGE STUDENT ACCOUNT* module of the use case diagram.

Use Case Name: CREATE STUDENT

Purpose: Record a new student account in the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `student : Student`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.CreateStudent` (POST).
- Appears in the *MANAGE STUDENT ACCOUNT* module of the use case diagram.

Use Case Name: UPDATE STUDENT

Purpose: Amend the stored details of an existing student account.

Actors:

- Admin (Primary Actor)

Input Parameters:

- `student : Student`
- `newPassword : string?`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.UpdateStudent` (POST).
- Appears in the *MANAGE STUDENT ACCOUNT* module of the use case diagram.

Use Case Name: DELETE STUDENT

Purpose: Remove a student account from the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `AdminController.DeleteStudent` (POST).
- Appears in the *MANAGE STUDENT ACCOUNT* module of the use case diagram.

Use Case Name: BULK CREATE STUDENTS

Purpose: Create many student accounts in one operation from pasted or uploaded CSV rows.

Actors:

- Admin (Primary Actor)

Input Parameters:

- `bulkFirstNames : List<string>?`
- `bulkLastNames : List<string>?`
- `bulkUserNames : List<string>?`
- `bulkPasswords : List<string>?`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.BulkCreateStudents` (POST).
- Appears in the *MANAGE STUDENT ACCOUNT* module of the use case diagram.
- Pulls in **BULK PREVIEW CSV** (<<include>>).

Use Case Name: BULK PREVIEW CSV

Purpose: Parse the submitted CSV and show what would be created, so mistakes are caught before any account exists.

Actors:

- Admin (Primary Actor)
- None; this behaviour runs inside **BULK CREATE STUDENTS** (Secondary Actor)

Input Parameters:

- `classId: int`
- `file: IFormFile?`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.
- **BULK CREATE STUDENTS** has reached the point where this is always performed.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.

5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.BulkPreviewCsv` (POST).
- Appears in the *MANAGE STUDENT ACCOUNT* module of the use case diagram.
- Drawn as <<include>> from **BULK CREATE STUDENTS**.

Manage Computer Profile AdminController

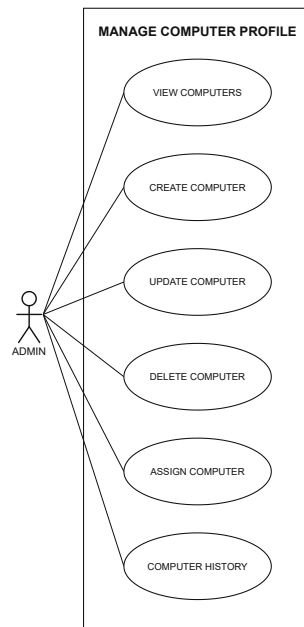


Figure 3.6: System Use Case for Manage Computer Profile

Use Case Name: VIEW COMPUTERS

Purpose: List the computers the signed-in user is allowed to see.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.Computers (GET)`.
- Appears in the *MANAGE COMPUTER PROFILE* module of the use case diagram.

Use Case Name: CREATE COMPUTER

Purpose: Record a new workstation profile in the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `computer : Computer`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.CreateComputer` (POST).
- Appears in the *MANAGE COMPUTER PROFILE* module of the use case diagram.

Use Case Name: UPDATE COMPUTER

Purpose: Amend the stored details of an existing workstation profile.

Actors:

- Admin (Primary Actor)

Input Parameters: `computer : Computer`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.UpdateComputer` (POST).
- Appears in the *MANAGE COMPUTER PROFILE* module of the use case diagram.

Use Case Name: DELETE COMPUTER

Purpose: Remove a workstation profile from the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `AdminController.DeleteComputer` (POST).
- Appears in the *MANAGE COMPUTER PROFILE* module of the use case diagram.

Use Case Name: ASSIGN COMPUTER

Purpose: Bind a workstation to a student so the workstation is recognised when that student signs in at it.

Actors:

- Admin (Primary Actor)

Input Parameters:

- `studentId: int`
- `computerId: int?`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.AssignComputer` (POST).
- Appears in the *MANAGE COMPUTER PROFILE* module of the use case diagram.

Use Case Name: COMPUTER HISTORY

Purpose: Show the recorded status changes for a workstation, and who made each one.

Actors:

- Admin (Primary Actor)

Input Parameters: `id:int`

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.ComputerHistory` (GET).
- Appears in the *MANAGE COMPUTER PROFILE* module of the use case diagram.

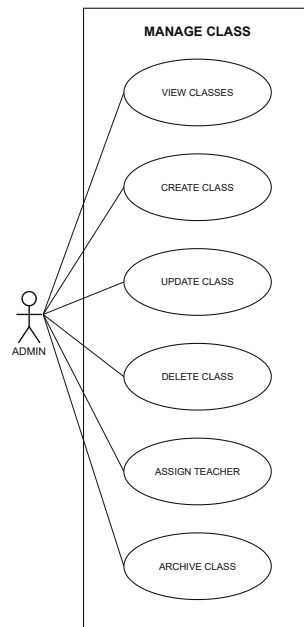
Manage Class AdminController

Figure 3.7: System Use Case for Manage Class

Use Case Name: VIEW CLASSES

Purpose: List the classes the signed-in user is allowed to see.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.Classes` (GET).
- Appears in the *MANAGE CLASS* module of the use case diagram.

Use Case Name: CREATE CLASS

Purpose: Record a new class in the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `cls : Class`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.CreateClass` (POST).
- Appears in the *MANAGE CLASS* module of the use case diagram.

Use Case Name: UPDATE CLASS

Purpose: Amend the stored details of an existing class.

Actors:

- Admin (Primary Actor)

Input Parameters: `cls : Class`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.UpdateClass` (POST).
- Appears in the *MANAGE CLASS* module of the use case diagram.

Use Case Name: DELETE CLASS

Purpose: Remove a class from the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `classId: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `AdminController.DeleteClass` (POST).
- Appears in the *MANAGE CLASS* module of the use case diagram.

Use Case Name: ASSIGN TEACHER

Purpose: Put a teacher in charge of a class.

Actors:

- Admin (Primary Actor)

Input Parameters:

- `classId: int`
- `teacherId: int?`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.AssignTeacher` (POST).
- Appears in the *MANAGE CLASS* module of the use case diagram.

Use Case Name: ARCHIVE CLASS

Purpose: Take a class out of active use while keeping its roster and records.

Actors:

- Admin (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.ArchiveClass` (POST).
- Appears in the *MANAGE CLASS* module of the use case diagram.

Manage Class Roster `AdminController`

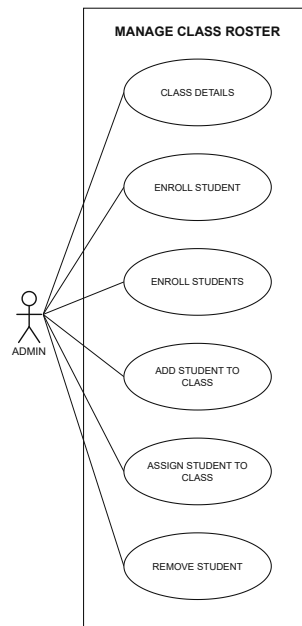


Figure 3.8: System Use Case for Manage Class Roster

Use Case Name: CLASS DETAILS

Purpose: Show one class with its roster and the students enrolled in it.

Actors:

- Admin (Primary Actor)

Input Parameters: `id:int`

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.ClassDetails` (GET).
- Appears in the *MANAGE CLASS ROSTER* module of the use case diagram.

Use Case Name: ENROLL STUDENT

Purpose: Add an existing student to a class roster.

Actors:

- Admin (Primary Actor)

Input Parameters:

- `classId: int`
- `studentId: int`
- `moveStudent: bool` (optional)

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.EnrollStudent` (POST).
- Appears in the *MANAGE CLASS ROSTER* module of the use case diagram.

Use Case Name: ENROLL STUDENTS

Purpose: Add several existing students to a class roster in one operation.

Actors:

- Admin (Primary Actor)

Input Parameters:

- `classId: int`
- `studentIds: List<int>?`
- `moveStudent: bool` (optional)

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.EnrollStudents` (POST).
- Appears in the *MANAGE CLASS ROSTER* module of the use case diagram.

Use Case Name: ADD STUDENT TO CLASS

Purpose: Create a new student account and place it on a class roster in one step.

Actors:

- Admin (Primary Actor)

Input Parameters:

- `classId: int`
- `firstName: string`
- `lastName: string`
- `username: string?`
- `password: string?`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.

5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.AddStudentToClass` (POST).
- Appears in the *MANAGE CLASS ROSTER* module of the use case diagram.

Use Case Name: ASSIGN STUDENT TO CLASS

Purpose: Set the primary class a student belongs to.

Actors:

- Admin (Primary Actor)

Input Parameters:

- `studentId: int`
- `classId: int?`
- `moveStudent: bool` (optional)

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.AssignStudentToClass` (POST).
- Appears in the *MANAGE CLASS ROSTER* module of the use case diagram.

Use Case Name: REMOVE STUDENT

Purpose: Take a student off a class roster while leaving the student account intact.

Actors:

- Admin (Primary Actor)

Input Parameters:

- `classId: int`
- `studentId: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.RemoveStudent (POST)`.
- Appears in the *MANAGE CLASS ROSTER* module of the use case diagram.

Manage Restriction Rule `AdminController`

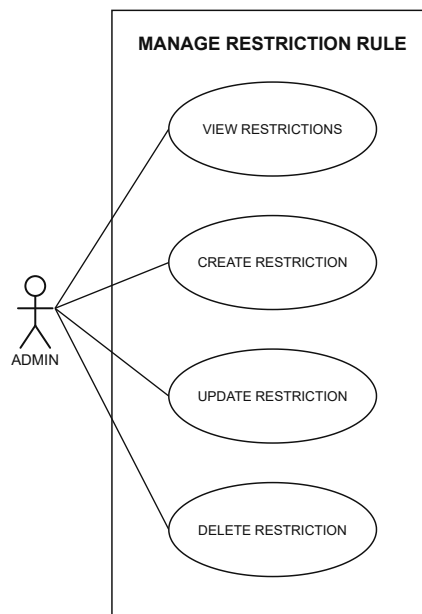


Figure 3.9: System Use Case for Manage Restriction Rule

Use Case Name: VIEW RESTRICTIONS

Purpose: List the restrictions the signed-in user is allowed to see.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.Restrictions (GET)`.
- Appears in the *MANAGE RESTRICTION RULE* module of the use case diagram.

Use Case Name: CREATE RESTRICTION

Purpose: Record a new restriction rule in the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `rule : RestrictionRule`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.CreateRestriction` (POST).
- Appears in the *MANAGE RESTRICTION RULE* module of the use case diagram.

Use Case Name: UPDATE RESTRICTION

Purpose: Amend the stored details of an existing restriction rule.

Actors:

- Admin (Primary Actor)

Input Parameters: `input : RestrictionRule`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.UpdateRestriction` (POST).
- Appears in the *MANAGE RESTRICTION RULE* module of the use case diagram.

Use Case Name: DELETE RESTRICTION

Purpose: Remove a restriction rule from the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `AdminController.DeleteRestriction (POST)`.
- Appears in the *MANAGE RESTRICTION RULE* module of the use case diagram.

Manage Blacklist And Whitelist AdminController

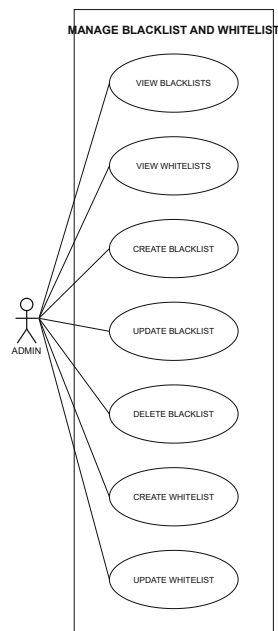


Figure 3.10: System Use Case for Manage Blacklist And Whitelist

Use Case Name: VIEW BLACKLISTS

Purpose: List the blacklists the signed-in user is allowed to see.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.Blacklists (GET)`.
- Appears in the *MANAGE BLACKLIST AND WHITELIST* module of the use case diagram.

Use Case Name: VIEW WHITELISTS

Purpose: List the whitelists the signed-in user is allowed to see.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.Whitelists (GET)`.
- Appears in the *MANAGE BLACKLIST AND WHITELIST* module of the use case diagram.

Use Case Name: CREATE BLACKLIST

Purpose: Record a new blacklist entry in the system.

Actors:

- Admin (Primary Actor)

Input Parameters:

- `RestrictionRuleId` — bound from the submitted form
- `RuleType` — bound from the submitted form
- `Target` — bound from the submitted form
- `Description` — bound from the submitted form
- `IsGlobal` — bound from the submitted form
- `IsActive` — bound from the submitted form
- `item:BlacklistItem`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.

4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.CreateBlacklist` (POST).
- Appears in the *MANAGE BLACKLIST AND WHITELIST* module of the use case diagram.

Use Case Name: UPDATE BLACKLIST

Purpose: Amend the stored details of an existing blacklist entry.

Actors:

- Admin (Primary Actor)

Input Parameters: `input : BlacklistItem`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.UpdateBlacklist` (POST).
- Appears in the *MANAGE BLACKLIST AND WHITELIST* module of the use case diagram.

Use Case Name: DELETE BLACKLIST

Purpose: Remove a blacklist entry from the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `AdminController.DeleteBlacklist` (POST).
- Appears in the *MANAGE BLACKLIST AND WHITELIST* module of the use case diagram.

Use Case Name: CREATE WHITELIST

Purpose: Record a new whitelist entry in the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `rule : RestrictionRule`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.CreateWhitelist` (POST).
- Appears in the *MANAGE BLACKLIST AND WHITELIST* module of the use case diagram.

Use Case Name: UPDATE WHITELIST

Purpose: Amend the stored details of an existing whitelist entry.

Actors:

- Admin (Primary Actor)

Input Parameters:

- `RuleType` — bound from the submitted form
- `Target` — bound from the submitted form
- `Description` — bound from the submitted form
- `IsGlobal` — bound from the submitted form
- `IsActive` — bound from the submitted form
- `rule:RestrictionRule`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.

5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.UpdateWhitelist` (POST).
- Appears in the *MANAGE BLACKLIST AND WHITELIST* module of the use case diagram.

Manage Category `AdminController`

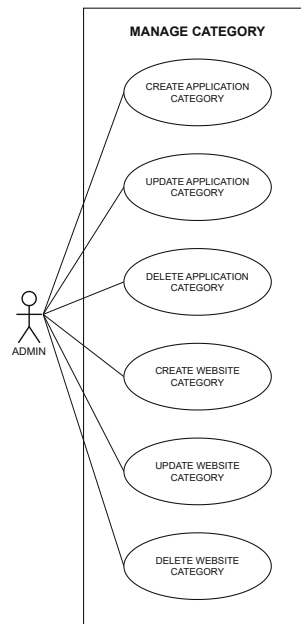


Figure 3.11: System Use Case for Manage Category

Use Case Name: CREATE APPLICATION CATEGORY

Purpose: Record a new application category in the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `category : ApplicationCategory`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.CreateApplicationCategory` (POST).
- Appears in the *MANAGE CATEGORY* module of the use case diagram.

Use Case Name: UPDATE APPLICATION CATEGORY

Purpose: Amend the stored details of an existing application category.

Actors:

- Admin (Primary Actor)

Input Parameters: `input : ApplicationCategory`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.UpdateApplicationCategory` (POST).
- Appears in the *MANAGE CATEGORY* module of the use case diagram.

Use Case Name: DELETE APPLICATION CATEGORY

Purpose: Remove a application category from the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `AdminController.DeleteApplicationCategory` (POST).
- Appears in the *MANAGE CATEGORY* module of the use case diagram.

Use Case Name: CREATE WEBSITE CATEGORY

Purpose: Record a new website category in the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `category : WebsiteCategory`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.CreateWebsiteCategory` (POST).
- Appears in the *MANAGE CATEGORY* module of the use case diagram.

Use Case Name: UPDATE WEBSITE CATEGORY

Purpose: Amend the stored details of an existing website category.

Actors:

- Admin (Primary Actor)

Input Parameters: `input : WebsiteCategory`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.UpdateWebsiteCategory` (POST).
- Appears in the *MANAGE CATEGORY* module of the use case diagram.

Use Case Name: DELETE WEBSITE CATEGORY

Purpose: Remove a website category from the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `AdminController.DeleteWebsiteCategory (POST)`.
- Appears in the *MANAGE CATEGORY* module of the use case diagram.

Manage Session Rule `AdminController`

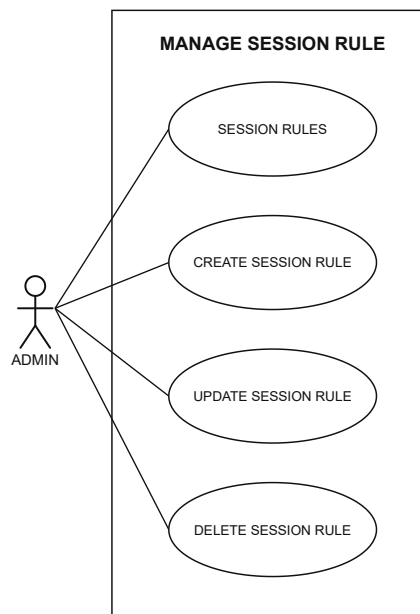


Figure 3.12: System Use Case for Manage Session Rule

Use Case Name: SESSION RULES

Purpose: List the session rules the signed-in user is allowed to see.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.SessionRules` (GET).
- Appears in the *MANAGE SESSION RULE* module of the use case diagram.

Use Case Name: CREATE SESSION RULE

Purpose: Record a new session rule in the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `rule : SessionRule`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.CreateSessionRule` (POST).
- Appears in the *MANAGE SESSION RULE* module of the use case diagram.

Use Case Name: UPDATE SESSION RULE

Purpose: Amend the stored details of an existing session rule.

Actors:

- Admin (Primary Actor)

Input Parameters: `input : SessionRule`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.UpdateSessionRule` (POST).
- Appears in the *MANAGE SESSION RULE* module of the use case diagram.

Use Case Name: DELETE SESSION RULE

Purpose: Remove a session rule from the system.

Actors:

- Admin (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `AdminController.DeleteSessionRule` (POST).
- Appears in the *MANAGE SESSION RULE* module of the use case diagram.

Manage Role And Permission `AdminController`

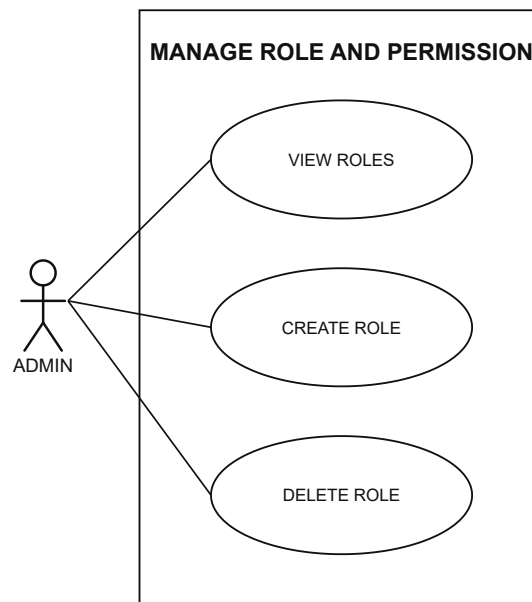


Figure 3.13: System Use Case for Manage Role And Permission

Use Case Name: VIEW ROLES

Purpose: Show the seeded roles and the permissions attached to them.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as an administrator or a teacher.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.Roles (GET)`.
- Appears in the *MANAGE ROLE AND PERMISSION* module of the use case diagram.

Use Case Name: CREATE ROLE

Purpose: Add a role to the seeded reference data.

Actors:

- Admin (Primary Actor)

Input Parameters: `role : Role`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as an administrator or a teacher.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.CreateRole` (POST).
- Appears in the *MANAGE ROLE AND PERMISSION* module of the use case diagram.

Use Case Name: DELETE ROLE

Purpose: Remove a role from the seeded reference data.

Actors:

- Admin (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.

- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `AdminController.DeleteRole (POST)`.
- Appears in the *MANAGE ROLE AND PERMISSION* module of the use case diagram.

Control Laboratory Session `AdminController`

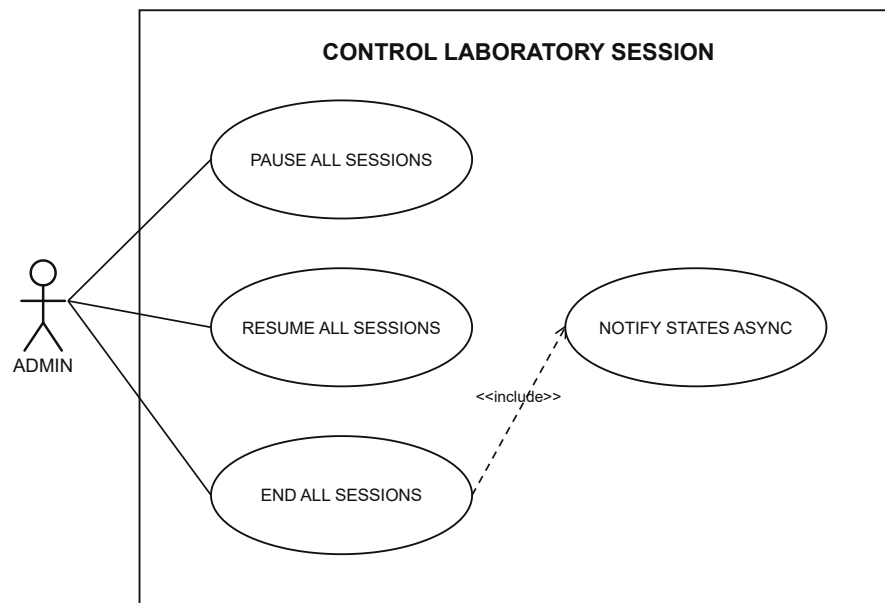


Figure 3.14: System Use Case for Control Laboratory Session

Use Case Name: PAUSE ALL SESSIONS

Purpose: Pause every active laboratory session at once, freezing the timers across the room.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.PauseAllSessions` (POST).
- Appears in the *CONTROL LABORATORY SESSION* module of the use case diagram.

Use Case Name: RESUME ALL SESSIONS

Purpose: Resume every paused laboratory session at once.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.ResumeAllSessions` (POST).
- Appears in the *CONTROL LABORATORY SESSION* module of the use case diagram.

Use Case Name: END ALL SESSIONS

Purpose: End every active laboratory session at once and tell the workstations.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.EndAllSessions` (POST).
- Appears in the *CONTROL LABORATORY SESSION* module of the use case diagram.
- Pulls in **NOTIFY STATES ASYNC** (<<include>>).

Use Case Name: NOTIFY STATES ASYNC

Purpose: Push the new session state to every affected workstation so clients stop their timers rather than waiting to notice.

Actors:

- Admin (Primary Actor)
- None; this behaviour runs inside **END ALL SESSIONS** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The behaviour completes and its effect is visible to the use case that includes it.

Pre-Condition:

- The caller is signed in.
- **END ALL SESSIONS** has reached the point where this is always performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The including use case reaches the point where this behaviour is required.
2. The server runs `NotifyStatesAsync` and applies its result.
3. Control returns to the including use case, which continues.

Exception Scenario:

- The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.

Additional Remarks:

- Implemented by `NotifyStatesAsync`.
- Appears in the *CONTROL LABORATORY SESSION* module of the use case diagram.
- Drawn as <<include>> from **END ALL SESSIONS**.

View Reports And Logs AdminController

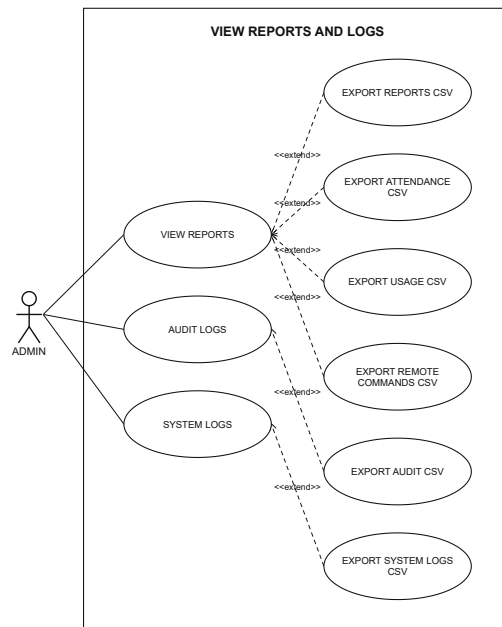


Figure 3.15: System Use Case for View Reports And Logs

Use Case Name: VIEW REPORTS

Purpose: Show the laboratory reports an administrator uses for oversight.

Actors:

- Admin (Primary Actor)

Input Parameters:

- `from : DateTime?`
- `to : DateTime?`
- `classId : int? (optional)`
- `station : string? (optional)`
- `page : int (optional)`
- `pageSize : int (optional)`

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher.
- The record named by the identifier exists.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.Reports` (GET).
- Appears in the *VIEW REPORTS AND LOGS* module of the use case diagram.
- Pulls in **EXPORT REPORTS CSV** (<<extend>>), **EXPORT ATTENDANCE CSV** (<<extend>>), **EXPORT USAGE CSV** (<<extend>>), **EXPORT REMOTE COMMANDS CSV** (<<extend>>).

Use Case Name: AUDIT LOGS

Purpose: Show the audit trail of administrative actions.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as an administrator or a teacher.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.AuditLogs (GET)`.
- Appears in the *VIEW REPORTS AND LOGS* module of the use case diagram.
- Pulls in **EXPORT AUDIT CSV** (<<extend>>).

Use Case Name: SYSTEM LOGS

Purpose: Show the technical log the server writes.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as an administrator or a teacher.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.SystemLogs (GET)`.
- Appears in the *VIEW REPORTS AND LOGS* module of the use case diagram.
- Pulls in **EXPORT SYSTEM LOGS CSV** (<<extend>>).

Use Case Name: EXPORT REPORTS CSV

Purpose: Produce the reports view as a CSV file the user can download.

Actors:

- Admin (Primary Actor)
- None; this behaviour runs inside **VIEW REPORTS** (Secondary Actor)

Input Parameters:

- `from:DateTime?` (optional)
- `to:DateTime?` (optional)
- `classId:int?` (optional)
- `station:string?` (optional)

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher.
- The record named by the identifier exists.
- **VIEW REPORTS** has reached the point where this is optionally performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.ExportReportsCsv` (GET).
- Appears in the *VIEW REPORTS AND LOGS* module of the use case diagram.
- Drawn as <<extend>> to **VIEW REPORTS**.

Use Case Name: EXPORT ATTENDANCE CSV

Purpose: Produce the attendance view as a CSV file the user can download.

Actors:

- Admin (Primary Actor)
- None; this behaviour runs inside **VIEW REPORTS** (Secondary Actor)

Input Parameters:

- `from: DateTime?` (optional)
- `to: DateTime?` (optional)
- `classId: int?` (optional)

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher.
- The record named by the identifier exists.
- **VIEW REPORTS** has reached the point where this is optionally performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.ExportAttendanceCsv` (GET).
- Appears in the *VIEW REPORTS AND LOGS* module of the use case diagram.
- Drawn as <<extend>> to **VIEW REPORTS**.

Use Case Name: EXPORT USAGE CSV

Purpose: Produce the usage view as a CSV file the user can download.

Actors:

- Admin (Primary Actor)
- None; this behaviour runs inside **VIEW REPORTS** (Secondary Actor)

Input Parameters:

- `from : DateTime?`
- `to : DateTime?`

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher.
- **VIEW REPORTS** has reached the point where this is optionally performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.ExportUsageCsv` (GET).
- Appears in the *VIEW REPORTS AND LOGS* module of the use case diagram.
- Drawn as <<extend>> to **VIEW REPORTS**.

Use Case Name: EXPORT REMOTE COMMANDS CSV

Purpose: Produce the remote commands view as a CSV file the user can download.

Actors:

- Admin (Primary Actor)
- None; this behaviour runs inside **VIEW REPORTS** (Secondary Actor)

Input Parameters:

- `from: DateTime?` (optional)
- `to: DateTime?` (optional)
- `teacherId: int?` (optional)

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher.
- The record named by the identifier exists.
- **VIEW REPORTS** has reached the point where this is optionally performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.ExportRemoteCommandsCsv (GET)`.
- Appears in the *VIEW REPORTS AND LOGS* module of the use case diagram.
- Drawn as <<extend>> to **VIEW REPORTS**.

Use Case Name: EXPORT AUDIT CSV

Purpose: Produce the audit view as a CSV file the user can download.

Actors:

- Admin (Primary Actor)
- None; this behaviour runs inside **AUDIT LOGS** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher.
- **AUDIT LOGS** has reached the point where this is optionally performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.ExportAuditCsv` (GET).
- Appears in the *VIEW REPORTS AND LOGS* module of the use case diagram.
- Drawn as <<extend>> to **AUDIT LOGS**.

Use Case Name: EXPORT SYSTEM LOGS CSV

Purpose: Produce the system logs view as a CSV file the user can download.

Actors:

- Admin (Primary Actor)
- None; this behaviour runs inside **SYSTEM LOGS** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher.
- **SYSTEM LOGS** has reached the point where this is optionally performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.ExportSystemLogsCsv` (GET).
- Appears in the *VIEW REPORTS AND LOGS* module of the use case diagram.
- Drawn as <<extend>> to **SYSTEM LOGS**.

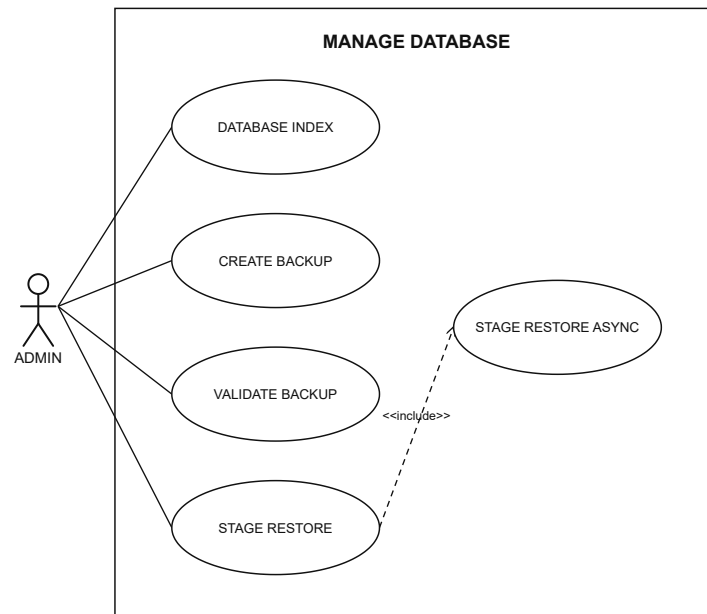
Manage Database AdminDatabaseController

Figure 3.16: System Use Case for Manage Database

Use Case Name: DATABASE INDEX

Purpose: Show database health: file size, integrity check, applied and pending migrations, and the backups on disk.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as an administrator.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminDatabaseController.Index (GET)`.
- Appears in the *MANAGE DATABASE* module of the use case diagram.

Use Case Name: CREATE BACKUP

Purpose: Take a backup copy of the SQLite database file and store it on the server.

Actors:

- Admin (Primary Actor)

Input Parameters: `label : string?`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as an administrator.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminDatabaseController.CreateBackup` (POST).
- Appears in the *MANAGE DATABASE* module of the use case diagram.

Use Case Name: VALIDATE BACKUP

Purpose: Check that a backup file is a readable, intact database before anybody relies on it.

Actors:

- Admin (Primary Actor)

Input Parameters: `backupFileName : string`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as an administrator.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminDatabaseController.ValidateBackup` (POST).
- Appears in the *MANAGE DATABASE* module of the use case diagram.

Use Case Name: STAGE RESTORE

Purpose: Stage a backup so the server restores it on the next restart, rather than swapping the file underneath a running system.

Actors:

- Admin (Primary Actor)

Input Parameters:

- `backupFileName : string`
- `confirmation : string?`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as an administrator.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminDatabaseController.StageRestore` (POST).
- Appears in the *MANAGE DATABASE* module of the use case diagram.
- Pulls in **STAGE RESTORE ASYNC** (<<include>>).

Use Case Name: STAGE RESTORE ASYNC

Purpose: Take a safety backup of the current database and mark the chosen backup for restore at next start.

Actors:

- Admin (Primary Actor)
- None; this behaviour runs inside **STAGE RESTORE** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The behaviour completes and its effect is visible to the use case that includes it.

Pre-Condition:

- The caller is signed in.
- **STAGE RESTORE** has reached the point where this is always performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The including use case reaches the point where this behaviour is required.
2. The server runs `StageRestoreAsync` and applies its result.
3. Control returns to the including use case, which continues.

Exception Scenario:

- The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.

Additional Remarks:

- Implemented by `StageRestoreAsync`.
- Appears in the *MANAGE DATABASE* module of the use case diagram.
- Drawn as <<include>> from **STAGE RESTORE**.

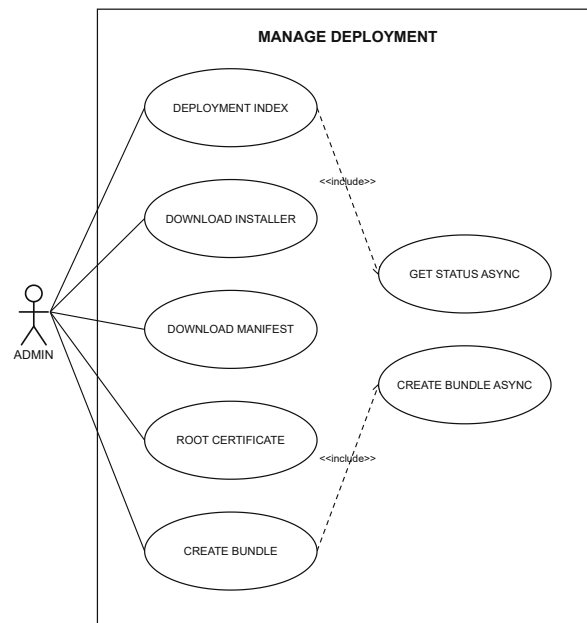
Manage Deployment AdminDeploymentController

Figure 3.17: System Use Case for Manage Deployment

Use Case Name: DEPLOYMENT INDEX

Purpose: Show the deployment hub: release version, installer and certificate state, and the endpoint clients should use.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as an administrator.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminDeploymentController.Index (GET)`.
- Appears in the *MANAGE DEPLOYMENT* module of the use case diagram.
- Pulls in **GET STATUS ASYNC** (<<include>>).

Use Case Name: DOWNLOAD INSTALLER

Purpose: Download the client installer for deployment to a workstation.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as an administrator.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminDeploymentController.Installer (GET)`.
- Appears in the *MANAGE DEPLOYMENT* module of the use case diagram.

Use Case Name: DOWNLOAD MANIFEST

Purpose: Download the release manifest listing the files and their hashes.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as an administrator.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminDeploymentController.Manifest (GET)`.
- Appears in the *MANAGE DEPLOYMENT* module of the use case diagram.

Use Case Name: ROOT CERTIFICATE

Purpose: Download the root certificate a workstation must trust to reach the server over HTTPS.

Actors:

- Admin (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as an administrator.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminDeploymentController.RootCertificate (GET)`.
- Appears in the *MANAGE DEPLOYMENT* module of the use case diagram.

Use Case Name: CREATE BUNDLE

Purpose: Build and download an offline client bundle for a workstation with no access to the server yet.

Actors:

- Admin (Primary Actor)

Input Parameters: `endpoint : string`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as an administrator.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Admin opens the page and CAMS confirms the role on the authentication cookie.
2. The Admin fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.

- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminDeploymentController.Bundle` (POST).
- Appears in the *MANAGE DEPLOYMENT* module of the use case diagram.
- Pulls in **CREATE BUNDLE ASYNC** (<<include>>).

Use Case Name: GET STATUS ASYNC

Purpose: Gather the deployment state: release version, file hashes, certificate and endpoint.

Actors:

- Admin (Primary Actor)
- None; this behaviour runs inside **DEPLOYMENT INDEX** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The behaviour completes and its effect is visible to the use case that includes it.

Pre-Condition:

- The caller is signed in.
- **DEPLOYMENT INDEX** has reached the point where this is always performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The including use case reaches the point where this behaviour is required.
2. The server runs `GetStatusAsync` and applies its result.
3. Control returns to the including use case, which continues.

Exception Scenario:

- The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.

Additional Remarks:

- Implemented by `GetStatusAsync`.
- Appears in the *MANAGE DEPLOYMENT* module of the use case diagram.
- Drawn as <<include>> from **DEPLOYMENT INDEX**.

Use Case Name: CREATE BUNDLE ASYNC

Purpose: Assemble an offline installer bundle carrying the installer, the root certificate and the server endpoint.

Actors:

- Admin (Primary Actor)
- None; this behaviour runs inside **CREATE BUNDLE** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The behaviour completes and its effect is visible to the use case that includes it.

Pre-Condition:

- The caller is signed in.
- **CREATE BUNDLE** has reached the point where this is always performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The including use case reaches the point where this behaviour is required.
2. The server runs `CreateBundleAsync` and applies its result.
3. Control returns to the including use case, which continues.

Exception Scenario:

- The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.

Additional Remarks:

- Implemented by `CreateBundleAsync`.
- Appears in the *MANAGE DEPLOYMENT* module of the use case diagram.
- Drawn as <<include>> from **CREATE BUNDLE**.

TEACHER

Process Log In AccountController

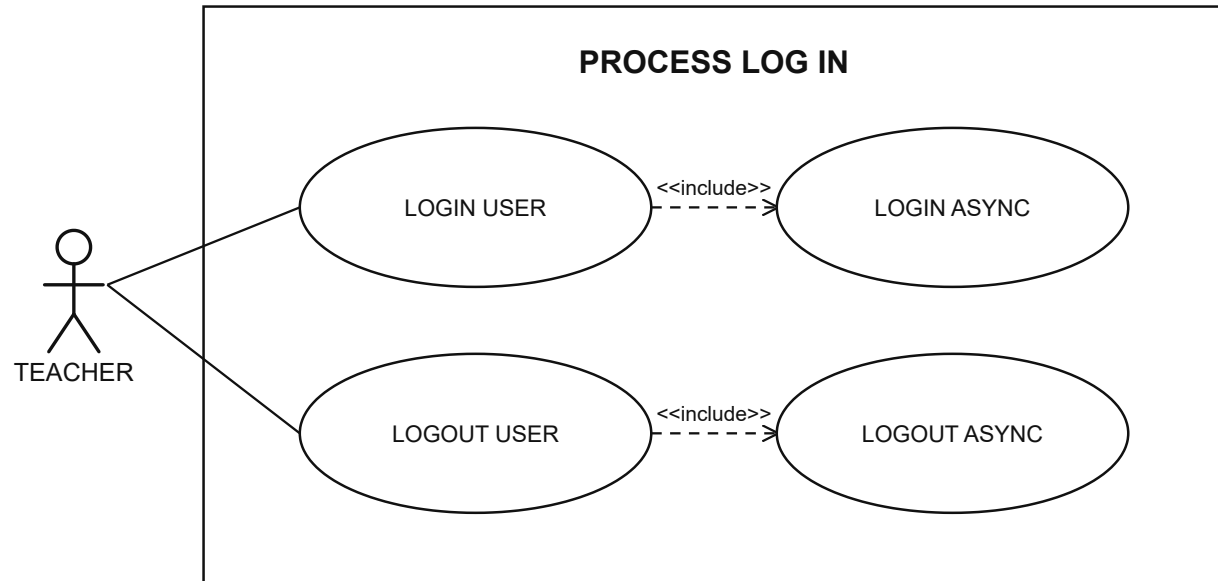


Figure 3.18: System Use Case for Process Log In

Use Case Name: LOGIN USER

Purpose: Let a person sign in to the CAMS web portal with a username and password, and place them in the part of the system their role allows.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `username : string`
- `password : string`

Output Parameters: An authentication cookie carrying the account role as a claim, and a redirect to the landing page for that role.

Pre-Condition: None. This is the sign-in endpoint and is reachable without an account session; the CAMS server must be running and reachable.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the CAMS sign-in page.
2. The Teacher enters a username and a password and submits the form.
3. The server validates the antiforgery token that accompanied the form.
4. The server checks the credentials against each account table in turn and finds the matching account.
5. The server confirms the account is active and not locked out.
6. The server issues an authentication cookie carrying the role as a claim.
7. The Teacher is redirected to the landing page for that role.

Exception Scenario:

- **The username matches no account** — the page reports that the sign-in failed, without saying which half was wrong.
- **The password does not match the stored hash** — the failed-attempt counter is raised and the same message is shown.
- **The account is locked out** — the sign-in is refused until the lockout expires, even with the right password.
- **The account is deactivated** — the sign-in is refused and the person is told to contact an administrator.

Additional Remarks:

- Implemented by `AccountController.Login` (POST).
- Appears in the *PROCESS LOG IN* module of the use case diagram.
- Pulls in **LOGIN ASYNC** (<<include>>).

Use Case Name: LOGOUT USER

Purpose: End the signed-in session and clear the authentication cookie, so the next visitor to the browser starts as an anonymous user.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The authentication cookie is cleared and the browser is returned to the sign-in page.

Pre-Condition: The caller is signed in.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher chooses to sign out.
2. The server closes any lab session the account still has open.
3. The server clears the authentication cookie.
4. The browser is returned to the sign-in page as an anonymous visitor.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AccountController.Logout` (POST).
- Appears in the *PROCESS LOG IN* module of the use case diagram.

- Pulls in **LOGOUT ASYNC** (<<include>>).

Use Case Name: LOGIN ASYNC

Purpose: Check the supplied password against the stored hash, confirm the account is active and not locked out, and decide which of the three account tables the person belongs to.

Actors:

- Teacher (Primary Actor)
- None; this behaviour runs inside **LOGIN USER** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: A `LoginResult` naming the role, the account id and the display name, or a result saying the credentials were rejected.

Pre-Condition:

- The sign-in use case has supplied a username and a password.
- **LOGIN USER** has reached the point where this is always performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The sign-in use case passes the username, the password, the workstation name and the caller address.
2. The service looks the username up in the administrator, teacher and student tables.
3. The stored hash is verified against the supplied password.
4. The active flag and the lockout expiry are checked.
5. On success the failed-attempt counter is cleared and the role and account id are returned; on failure the counter is raised.

Exception Scenario:

- The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.

Additional Remarks:

- Implemented by `LoginAsync`.
- Appears in the *PROCESS LOG IN* module of the use case diagram.
- Drawn as `<<include>>` from **LOGIN USER**.

Use Case Name: LOGOUT ASYNC

Purpose: Close any lab session the student still has open and record the sign-out.

Actors:

- Teacher (Primary Actor)
- None; this behaviour runs inside **LOGOUT USER** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The behaviour completes and its effect is visible to the use case that includes it.

Pre-Condition:

- The caller is signed in.
- **LOGOUT USER** has reached the point where this is always performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The including use case reaches the point where this behaviour is required.
2. The server runs `LogoutAsync` and applies its result.
3. Control returns to the including use case, which continues.

Exception Scenario:

- The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.

Additional Remarks:

- Implemented by `LogoutAsync`.
- Appears in the *PROCESS LOG IN* module of the use case diagram.
- Drawn as <<include>> from **LOGOUT USER**.

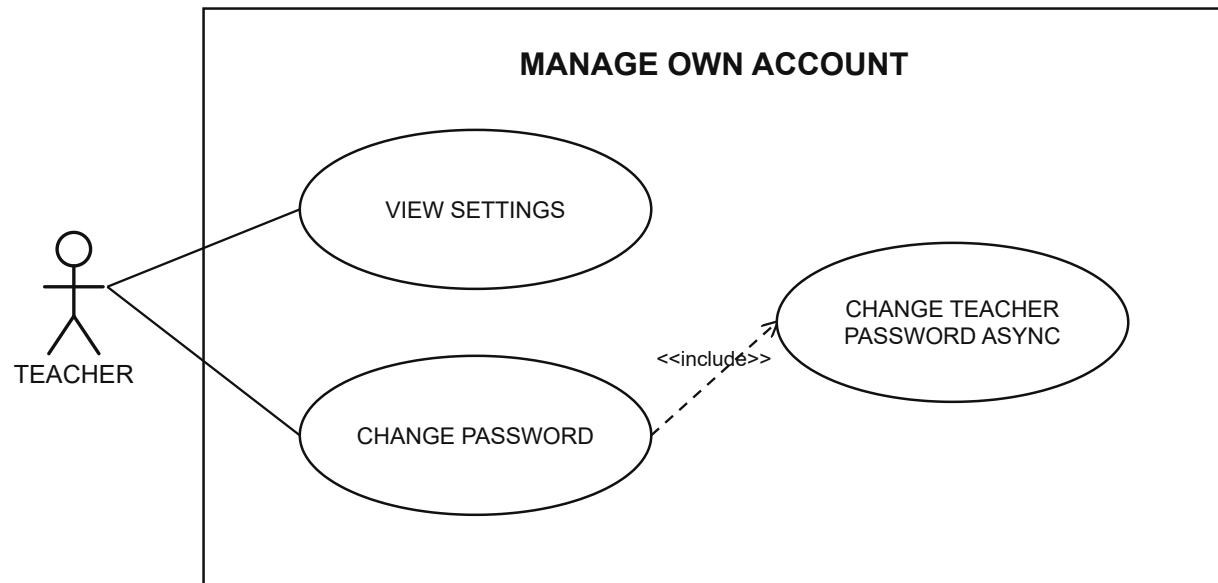
Manage Own Account TeacherController

Figure 3.19: System Use Case for Manage Own Account

Use Case Name: VIEW SETTINGS

Purpose: Show the settings page for the signed-in user.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `TeacherController.Settings` (GET).
- Appears in the *MANAGE OWN ACCOUNT* module of the use case diagram.

Use Case Name: CHANGE PASSWORD

Purpose: Let the signed-in user replace their own password after proving they know the current one.

Actors:

- Teacher (Primary Actor)

Input Parameters: `input : PasswordChangeInput`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.

- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `TeacherController.ChangePassword` (POST).
- Appears in the *MANAGE OWN ACCOUNT* module of the use case diagram.
- Pulls in **CHANGE TEACHER PASSWORD ASYNC** (<<include>>).

Use Case Name: CHANGE TEACHER PASSWORD ASYNC

Purpose: Verify the current password and replace the stored hash for a teacher account.

Actors:

- Teacher (Primary Actor)
- None; this behaviour runs inside **CHANGE PASSWORD** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The behaviour completes and its effect is visible to the use case that includes it.

Pre-Condition:

- The caller is signed in.
- **CHANGE PASSWORD** has reached the point where this is always performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The including use case reaches the point where this behaviour is required.
2. The server runs `ChangeTeacherPasswordAsync` and applies its result.
3. Control returns to the including use case, which continues.

Exception Scenario:

- The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.

Additional Remarks:

- Implemented by `ChangeTeacherPasswordAsync`.
- Appears in the *MANAGE OWN ACCOUNT* module of the use case diagram.
- Drawn as <<include>> from **CHANGE PASSWORD**.

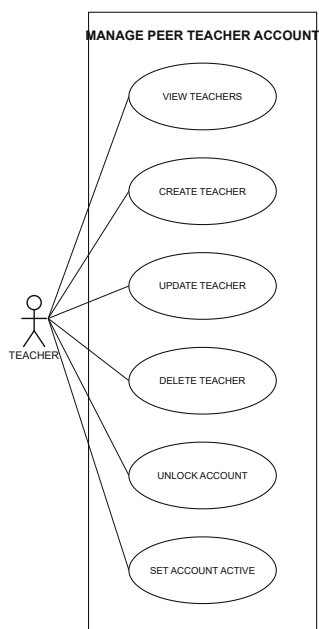
Manage Peer Teacher Account AdminController

Figure 3.20: System Use Case for Manage Peer Teacher Account

Use Case Name: VIEW TEACHERS

Purpose: List the teachers the signed-in user is allowed to see.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.Teachers (GET)`.
- Appears in the *MANAGE PEER TEACHER ACCOUNT* module of the use case diagram.

Use Case Name: CREATE TEACHER

Purpose: Record a new teacher account in the system.

Actors:

- Teacher (Primary Actor)

Input Parameters: `teacher : Teacher`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.CreateTeacher` (POST).
- Appears in the *MANAGE PEER TEACHER ACCOUNT* module of the use case diagram.

Use Case Name: UPDATE TEACHER

Purpose: Amend the stored details of an existing teacher account.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `teacher : Teacher`
- `newPassword : string?`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.UpdateTeacher` (POST).
- Appears in the *MANAGE PEER TEACHER ACCOUNT* module of the use case diagram.

Use Case Name: DELETE TEACHER

Purpose: Remove a teacher account from the system.

Actors:

- Teacher (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `AdminController.DeleteTeacher` (POST).
- Appears in the *MANAGE PEER TEACHER ACCOUNT* module of the use case diagram.

Use Case Name: UNLOCK ACCOUNT

Purpose: Clear the lockout on an account that has been locked by repeated failed sign-in attempts.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `accountRole : AccountRole`
- `id : int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.

6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.UnlockAccount` (POST).
- Appears in the *MANAGE PEER TEACHER ACCOUNT* module of the use case diagram.

Use Case Name: SET ACCOUNT ACTIVE

Purpose: Activate or deactivate an account without deleting it, so a person can be kept out of the system while their records survive.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `accountRole : AccountRole`
- `id : int`
- `isActive : bool`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.

5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.SetAccountActive` (POST).
- Appears in the *MANAGE PEER TEACHER ACCOUNT* module of the use case diagram.

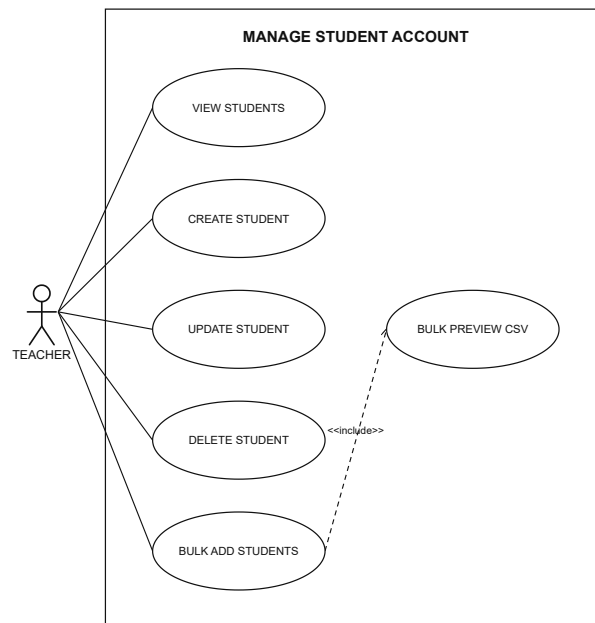
Manage Student Account TeacherController

Figure 3.21: System Use Case for Manage Student Account

Use Case Name: VIEW STUDENTS

Purpose: List the students the signed-in user is allowed to see.

Actors:

- Teacher (Primary Actor)

Input Parameters: `search : string?` (optional)

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `TeacherController.Students` (GET).
- Appears in the *MANAGE STUDENT ACCOUNT* module of the use case diagram.

Use Case Name: CREATE STUDENT

Purpose: Record a new student account in the system.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `student : Student`
- `classId : int?` (optional)
- `search : string?` (optional)

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.CreateStudent` (POST).
- Appears in the *MANAGE STUDENT ACCOUNT* module of the use case diagram.

Use Case Name: UPDATE STUDENT

Purpose: Amend the stored details of an existing student account.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `student : Student`
- `newPassword : string?`
- `search : string? (optional)`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `TeacherController.UpdateStudent` (POST).
- Appears in the *MANAGE STUDENT ACCOUNT* module of the use case diagram.

Use Case Name: DELETE STUDENT

Purpose: Remove a student account from the system.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `studentId: int`
- `search: string?` (optional)

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `TeacherController.DeleteStudent` (POST).
- Appears in the *MANAGE STUDENT ACCOUNT* module of the use case diagram.

Use Case Name: BULK ADD STUDENTS

Purpose: Create many student accounts at once and enrol them into a class in the same operation.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `classId: int`
- `bulkFirstNames: List<string>?`
- `bulkLastNames: List<string>?`
- `bulkUserNames: List<string>?`
- `bulkPasswords: List<string>?`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.

5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.BulkAddStudents` (POST).
- Appears in the *MANAGE STUDENT ACCOUNT* module of the use case diagram.
- Pulls in **BULK PREVIEW CSV** (<<include>>).

Use Case Name: BULK PREVIEW CSV

Purpose: Parse the submitted CSV and show what would be created, so mistakes are caught before any account exists.

Actors:

- Teacher (Primary Actor)
- None; this behaviour runs inside **BULK ADD STUDENTS** (Secondary Actor)

Input Parameters:

- `classId:int`
- `file:IFormFile?`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.
- **BULK ADD STUDENTS** has reached the point where this is always performed.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.

6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.BulkPreviewCsv` (POST).
- Appears in the *MANAGE STUDENT ACCOUNT* module of the use case diagram.
- Drawn as <<include>> from **BULK ADD STUDENTS**.

Manage Computer Profile AdminController

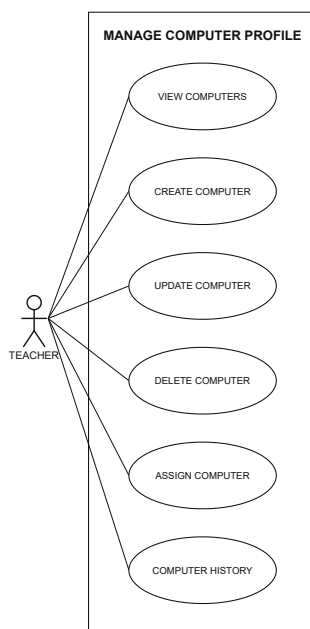


Figure 3.22: System Use Case for Manage Computer Profile

Use Case Name: VIEW COMPUTERS

Purpose: List the computers the signed-in user is allowed to see.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.Computers (GET)`.
- Appears in the *MANAGE COMPUTER PROFILE* module of the use case diagram.

Use Case Name: CREATE COMPUTER

Purpose: Record a new workstation profile in the system.

Actors:

- Teacher (Primary Actor)

Input Parameters: `computer : Computer`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.CreateComputer` (POST).
- Appears in the *MANAGE COMPUTER PROFILE* module of the use case diagram.

Use Case Name: UPDATE COMPUTER

Purpose: Amend the stored details of an existing workstation profile.

Actors:

- Teacher (Primary Actor)

Input Parameters: `computer : Computer`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.UpdateComputer` (POST).
- Appears in the *MANAGE COMPUTER PROFILE* module of the use case diagram.

Use Case Name: DELETE COMPUTER

Purpose: Remove a workstation profile from the system.

Actors:

- Teacher (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `AdminController.DeleteComputer` (POST).
- Appears in the *MANAGE COMPUTER PROFILE* module of the use case diagram.

Use Case Name: ASSIGN COMPUTER

Purpose: Bind a workstation to a student so the workstation is recognised when that student signs in at it.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `studentId:int`
- `computerId:int?`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.

6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.AssignComputer` (POST).
- Appears in the *MANAGE COMPUTER PROFILE* module of the use case diagram.

Use Case Name: COMPUTER HISTORY

Purpose: Show the recorded status changes for a workstation, and who made each one.

Actors:

- Teacher (Primary Actor)

Input Parameters: `id:int`

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `AdminController.ComputerHistory (GET)`.

- Appears in the *MANAGE COMPUTER PROFILE* module of the use case diagram.

Manage Class TeacherController

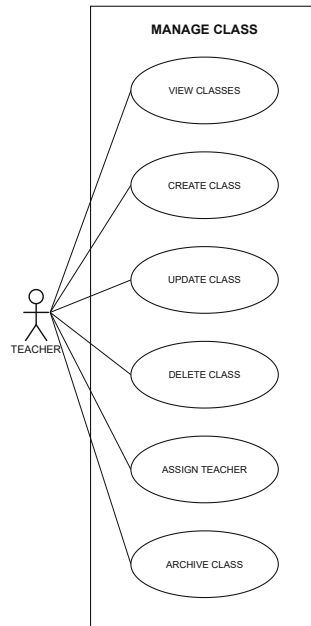


Figure 3.23: System Use Case for Manage Class

Use Case Name: VIEW CLASSES

Purpose: List the classes the signed-in user is allowed to see.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `TeacherController.Classes` (GET).
- Appears in the *MANAGE CLASS* module of the use case diagram.

Use Case Name: CREATE CLASS

Purpose: Record a new class in the system.

Actors:

- Teacher (Primary Actor)

Input Parameters: `cls : Class`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.

- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `TeacherController.CreateClass` (POST).
- Appears in the *MANAGE CLASS* module of the use case diagram.

Use Case Name: UPDATE CLASS

Purpose: Amend the stored details of an existing class.

Actors:

- Teacher (Primary Actor)

Input Parameters: `cls : Class`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.

- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `TeacherController.UpdateClass` (POST).
- Appears in the *MANAGE CLASS* module of the use case diagram.

Use Case Name: DELETE CLASS

Purpose: Remove a class from the system.

Actors:

- Teacher (Primary Actor)

Input Parameters: `classId: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `TeacherController.DeleteClass` (POST).
- Appears in the *MANAGE CLASS* module of the use case diagram.

Use Case Name: ASSIGN TEACHER

Purpose: Put a teacher in charge of a class.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `classId: int`
- `teacherId: int?`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.AssignTeacher` (POST).
- Appears in the *MANAGE CLASS* module of the use case diagram.

Use Case Name: ARCHIVE CLASS

Purpose: Take a class out of active use while keeping its roster and records.

Actors:

- Teacher (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.ArchiveClass` (POST).
- Appears in the *MANAGE CLASS* module of the use case diagram.

Manage Class Roster `TeacherController`

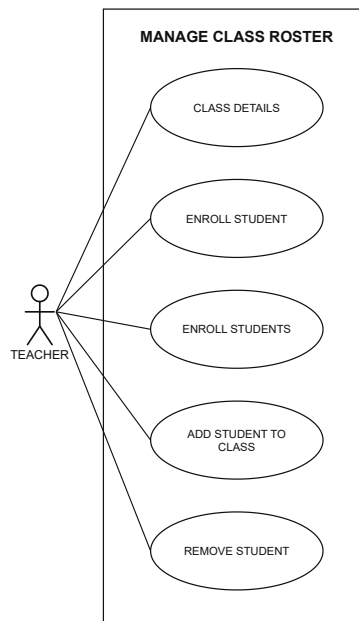


Figure 3.24: System Use Case for Manage Class Roster

Use Case Name: CLASS DETAILS

Purpose: Show one class with its roster and the students enrolled in it.

Actors:

- Teacher (Primary Actor)

Input Parameters: `id: int`

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.ClassDetails` (GET).
- Appears in the *MANAGE CLASS ROSTER* module of the use case diagram.

Use Case Name: ENROLL STUDENT

Purpose: Add an existing student to a class roster.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `classId: int`
- `studentId: int`
- `moveStudent: bool` (optional)

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.EnrollStudent` (POST).
- Appears in the *MANAGE CLASS ROSTER* module of the use case diagram.

Use Case Name: ENROLL STUDENTS

Purpose: Add several existing students to a class roster in one operation.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `classId: int`
- `studentIds: List<int>?`
- `moveStudent: bool` (optional)

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.EnrollStudents` (POST).
- Appears in the *MANAGE CLASS ROSTER* module of the use case diagram.

Use Case Name: ADD STUDENT TO CLASS

Purpose: Create a new student account and place it on a class roster in one step.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `classId: int`
- `firstName: string`
- `lastName: string`
- `username: string?`
- `password: string?`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.

6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.AddStudentToClass` (POST).
- Appears in the *MANAGE CLASS ROSTER* module of the use case diagram.

Use Case Name: REMOVE STUDENT

Purpose: Take a student off a class roster while leaving the student account intact.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `classId: int`
- `studentId: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.RemoveStudent (POST)`.
- Appears in the *MANAGE CLASS ROSTER* module of the use case diagram.

Manage Restriction Rule `TeacherController`

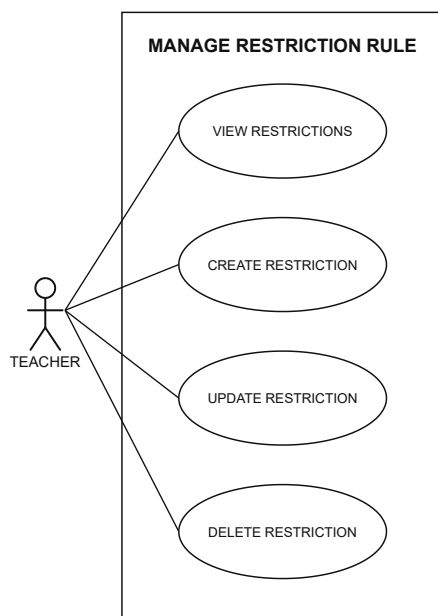


Figure 3.25: System Use Case for Manage Restriction Rule

Use Case Name: VIEW RESTRICTIONS

Purpose: List the restrictions the signed-in user is allowed to see.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `TeacherController.Restrictions (GET)`.
- Appears in the *MANAGE RESTRICTION RULE* module of the use case diagram.

Use Case Name: CREATE RESTRICTION

Purpose: Record a new restriction rule in the system.

Actors:

- Teacher (Primary Actor)

Input Parameters: `rule : RestrictionRule`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.

- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `TeacherController.CreateRestriction` (POST).
- Appears in the *MANAGE RESTRICTION RULE* module of the use case diagram.

Use Case Name: UPDATE RESTRICTION

Purpose: Amend the stored details of an existing restriction rule.

Actors:

- Teacher (Primary Actor)

Input Parameters: `input : RestrictionRule`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.

- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `TeacherController.UpdateRestriction` (POST).
- Appears in the *MANAGE RESTRICTION RULE* module of the use case diagram.

Use Case Name: DELETE RESTRICTION

Purpose: Remove a restriction rule from the system.

Actors:

- Teacher (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `TeacherController.DeleteRestriction (POST)`.
- Appears in the *MANAGE RESTRICTION RULE* module of the use case diagram.

Manage Blacklist And Whitelist AdminController

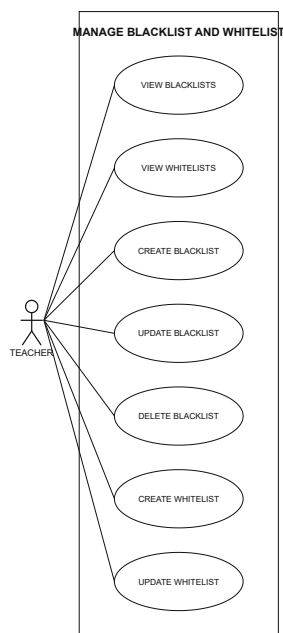


Figure 3.26: System Use Case for Manage Blacklist And Whitelist

Use Case Name: VIEW BLACKLISTS

Purpose: List the blacklists the signed-in user is allowed to see.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.Blacklists (GET)`.
- Appears in the *MANAGE BLACKLIST AND WHITELIST* module of the use case diagram.

Use Case Name: VIEW WHITELISTS

Purpose: List the whitelists the signed-in user is allowed to see.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.Whitelists (GET)`.
- Appears in the *MANAGE BLACKLIST AND WHITELIST* module of the use case diagram.

Use Case Name: CREATE BLACKLIST

Purpose: Record a new blacklist entry in the system.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `RestrictionRuleId` — bound from the submitted form
- `RuleType` — bound from the submitted form
- `Target` — bound from the submitted form
- `Description` — bound from the submitted form
- `IsGlobal` — bound from the submitted form
- `IsActive` — bound from the submitted form
- `item:BlacklistItem`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.

3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.CreateBlacklist` (POST).
- Appears in the *MANAGE BLACKLIST AND WHITELIST* module of the use case diagram.

Use Case Name: UPDATE BLACKLIST

Purpose: Amend the stored details of an existing blacklist entry.

Actors:

- Teacher (Primary Actor)

Input Parameters: `input : BlackListItem`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.UpdateBlacklist` (POST).
- Appears in the *MANAGE BLACKLIST AND WHITELIST* module of the use case diagram.

Use Case Name: DELETE BLACKLIST

Purpose: Remove a blacklist entry from the system.

Actors:

- Teacher (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `AdminController.DeleteBlacklist` (POST).
- Appears in the *MANAGE BLACKLIST AND WHITELIST* module of the use case diagram.

Use Case Name: CREATE WHITELIST

Purpose: Record a new whitelist entry in the system.

Actors:

- Teacher (Primary Actor)

Input Parameters: `rule : RestrictionRule`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.CreateWhitelist` (POST).
- Appears in the *MANAGE BLACKLIST AND WHITELIST* module of the use case diagram.

Use Case Name: UPDATE WHITELIST

Purpose: Amend the stored details of an existing whitelist entry.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `RuleType` — bound from the submitted form
- `Target` — bound from the submitted form
- `Description` — bound from the submitted form
- `IsGlobal` — bound from the submitted form
- `IsActive` — bound from the submitted form
- `rule:RestrictionRule`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.

4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.UpdateWhitelist` (POST).
- Appears in the *MANAGE BLACKLIST AND WHITELIST* module of the use case diagram.

Manage Category AdminController

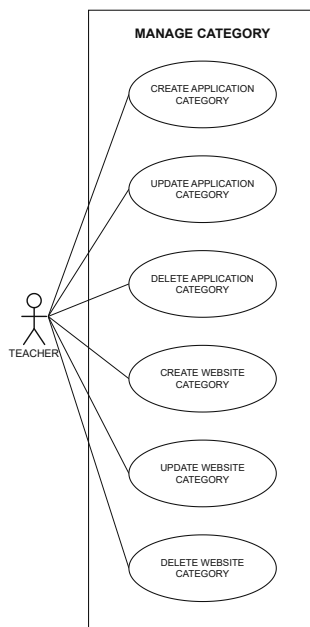


Figure 3.27: System Use Case for Manage Category

Use Case Name: CREATE APPLICATION CATEGORY

Purpose: Record a new application category in the system.

Actors:

- Teacher (Primary Actor)

Input Parameters: `category : ApplicationCategory`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.CreateApplicationCategory` (POST).
- Appears in the *MANAGE CATEGORY* module of the use case diagram.

Use Case Name: UPDATE APPLICATION CATEGORY

Purpose: Amend the stored details of an existing application category.

Actors:

- Teacher (Primary Actor)

Input Parameters: `input : ApplicationCategory`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.UpdateApplicationCategory` (POST).
- Appears in the *MANAGE CATEGORY* module of the use case diagram.

Use Case Name: DELETE APPLICATION CATEGORY

Purpose: Remove a application category from the system.

Actors:

- Teacher (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `AdminController.DeleteApplicationCategory` (POST).
- Appears in the *MANAGE CATEGORY* module of the use case diagram.

Use Case Name: CREATE WEBSITE CATEGORY

Purpose: Record a new website category in the system.

Actors:

- Teacher (Primary Actor)

Input Parameters: `category : WebsiteCategory`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.CreateWebsiteCategory` (POST).
- Appears in the *MANAGE CATEGORY* module of the use case diagram.

Use Case Name: UPDATE WEBSITE CATEGORY

Purpose: Amend the stored details of an existing website category.

Actors:

- Teacher (Primary Actor)

Input Parameters: `input : WebsiteCategory`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.UpdateWebsiteCategory` (POST).
- Appears in the *MANAGE CATEGORY* module of the use case diagram.

Use Case Name: DELETE WEBSITE CATEGORY

Purpose: Remove a website category from the system.

Actors:

- Teacher (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `AdminController.DeleteWebsiteCategory (POST)`.
- Appears in the *MANAGE CATEGORY* module of the use case diagram.

Manage Session Rule `AdminController`

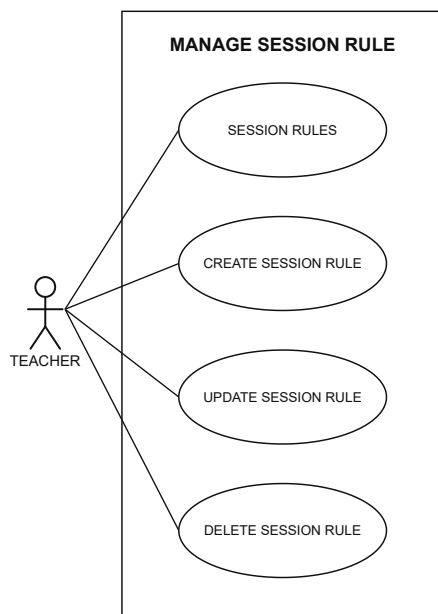


Figure 3.28: System Use Case for Manage Session Rule

Use Case Name: SESSION RULES

Purpose: List the session rules the signed-in user is allowed to see.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.SessionRules (GET)`.
- Appears in the *MANAGE SESSION RULE* module of the use case diagram.

Use Case Name: CREATE SESSION RULE

Purpose: Record a new session rule in the system.

Actors:

- Teacher (Primary Actor)

Input Parameters: `rule : SessionRule`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.CreateSessionRule` (POST).
- Appears in the *MANAGE SESSION RULE* module of the use case diagram.

Use Case Name: UPDATE SESSION RULE

Purpose: Amend the stored details of an existing session rule.

Actors:

- Teacher (Primary Actor)

Input Parameters: `input : SessionRule`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AdminController.UpdateSessionRule` (POST).
- Appears in the *MANAGE SESSION RULE* module of the use case diagram.

Use Case Name: DELETE SESSION RULE

Purpose: Remove a session rule from the system.

Actors:

- Teacher (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The record is still referenced** — the deletion is refused rather than leaving dangling references.

Additional Remarks:

- Implemented by `AdminController.DeleteSessionRule` (POST).
- Appears in the *MANAGE SESSION RULE* module of the use case diagram.

Control Laboratory Session `TeacherController`

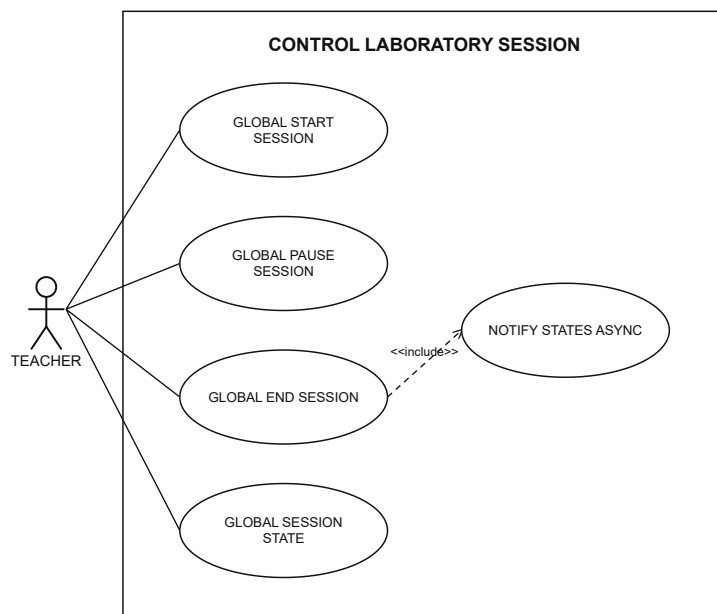


Figure 3.29: System Use Case for Control Laboratory Session

Use Case Name: GLOBAL START SESSION

Purpose: Start a laboratory-wide session so every connected workstation begins at the same moment.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.

- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `TeacherController.GlobalStartSession` (POST).
- Appears in the *CONTROL LABORATORY SESSION* module of the use case diagram.

Use Case Name: GLOBAL PAUSE SESSION

Purpose: Pause the laboratory-wide session for the whole room.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.

- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `TeacherController.GlobalPauseSession` (POST).
- Appears in the *CONTROL LABORATORY SESSION* module of the use case diagram.

Use Case Name: GLOBAL END SESSION

Purpose: End the laboratory-wide session for the whole room.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.

- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `TeacherController.GlobalEndSession` (POST).
- Appears in the *CONTROL LABORATORY SESSION* module of the use case diagram.
- Pulls in **NOTIFY STATES ASYNC** (<<include>>).

Use Case Name: GLOBAL SESSION STATE

Purpose: Report the current laboratory-wide session state so the page can show the right timer and controls.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `TeacherController.GlobalSessionState (GET)`.
- Appears in the *CONTROL LABORATORY SESSION* module of the use case diagram.

Use Case Name: NOTIFY STATES ASYNC

Purpose: Push the new session state to every affected workstation so clients stop their timers rather than waiting to notice.

Actors:

- Teacher (Primary Actor)
- None; this behaviour runs inside **GLOBAL END SESSION** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The behaviour completes and its effect is visible to the use case that includes it.

Pre-Condition:

- The caller is signed in.
- **GLOBAL END SESSION** has reached the point where this is always performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The including use case reaches the point where this behaviour is required.
2. The server runs `NotifyStatesAsync` and applies its result.
3. Control returns to the including use case, which continues.

Exception Scenario:

- The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.

Additional Remarks:

- Implemented by `NotifyStatesAsync`.
- Appears in the *CONTROL LABORATORY SESSION* module of the use case diagram.
- Drawn as <<include>> from **GLOBAL END SESSION**.

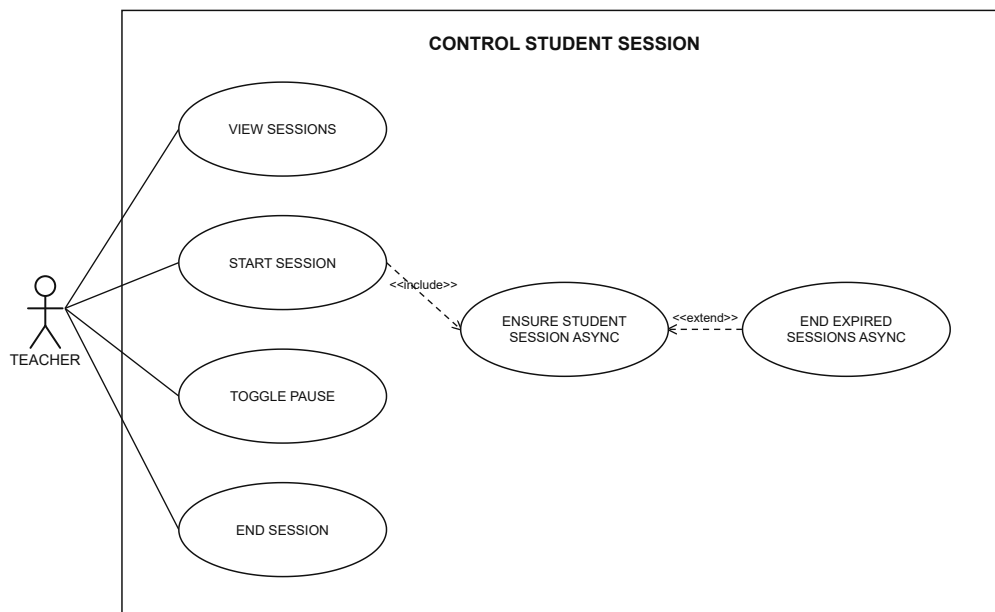
Control Student Session TeacherController

Figure 3.30: System Use Case for Control Student Session

Use Case Name: VIEW SESSIONS

Purpose: List the lab sessions the teacher may act on.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `TeacherController.Sessions` (GET).
- Appears in the *CONTROL STUDENT SESSION* module of the use case diagram.

Use Case Name: START SESSION

Purpose: Open a lab session for one student at one workstation under a chosen session rule.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `studentId: int`
- `computerId: int?`
- `sessionRuleId: int?`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.StartSession` (POST).
- Appears in the *CONTROL STUDENT SESSION* module of the use case diagram.
- Pulls in **ENSURE STUDENT SESSION ASYNC** (<<include>>).

Use Case Name: TOGGLE PAUSE

Purpose: Pause a running session, or resume a paused one, accumulating the paused time so it is not charged against the limit.

Actors:

- Teacher (Primary Actor)

Input Parameters: `id:int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.TogglePause` (POST).
- Appears in the *CONTROL STUDENT SESSION* module of the use case diagram.

Use Case Name: END SESSION

Purpose: Close one student lab session and record its end time.

Actors:

- Teacher (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.EndSession` (POST).
- Appears in the *CONTROL STUDENT SESSION* module of the use case diagram.

Use Case Name: ENSURE STUDENT SESSION ASYNC

Purpose: Find the student a session, creating one if none is open, and apply the governing session rule to it.

Actors:

- Teacher (Primary Actor)
- None; this behaviour runs inside **START SESSION** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The behaviour completes and its effect is visible to the use case that includes it.

Pre-Condition:

- The caller is signed in.
- **START SESSION** has reached the point where this is always performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The including use case reaches the point where this behaviour is required.
2. The server runs `EnsureStudentSessionAsync` and applies its result.
3. Control returns to the including use case, which continues.

Exception Scenario:

- The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.

Additional Remarks:

- Implemented by `EnsureStudentSessionAsync`.
- Appears in the *CONTROL STUDENT SESSION* module of the use case diagram.
- Drawn as <<include>> from **START SESSION**.
- Pulls in **END EXPIRED SESSIONS ASYNC** (<<extend>>).

Use Case Name: END EXPIRED SESSIONS ASYNC

Purpose: Close sessions that have run past the limit their session rule sets, without anyone having to intervene.

Actors:

- Teacher (Primary Actor)
- None; this behaviour runs inside **ENSURE STUDENT SESSION ASYNC** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The behaviour completes and its effect is visible to the use case that includes it.

Pre-Condition:

- The caller is signed in.
- **ENSURE STUDENT SESSION ASYNC** has reached the point where this is optionally performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The including use case reaches the point where this behaviour is required.
2. The server runs `EndExpiredSessionsAsync` and applies its result.
3. Control returns to the including use case, which continues.

Exception Scenario:

- The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.

Additional Remarks:

- Implemented by `EndExpiredSessionsAsync`.
- Appears in the *CONTROL STUDENT SESSION* module of the use case diagram.

- Drawn as <<extend>> to **ENSURE STUDENT SESSION ASYNC.**

Monitor Student Screen TeacherController + Hub

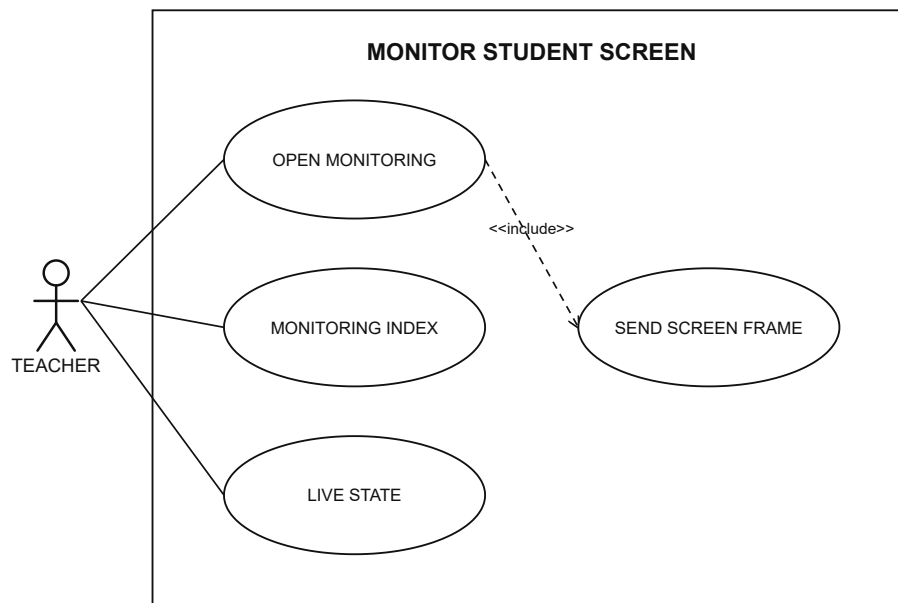


Figure 3.31: System Use Case for Monitor Student Screen

Use Case Name: OPEN MONITORING

Purpose: Open the live monitoring wall showing every connected student workstation.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `TeacherController.Monitoring (GET)`.
- Appears in the *MONITOR STUDENT SCREEN* module of the use case diagram.
- Pulls in **SEND SCREEN FRAME** (<<include>>).

Use Case Name: MONITORING INDEX

Purpose: Serve the monitoring surface the live screen grid is built on.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries `[TeacherSharedAction]`; the teacher account must also be active.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `AdminController.Index (GET)`.
- Appears in the *MONITOR STUDENT SCREEN* module of the use case diagram.

Use Case Name: LIVE STATE

Purpose: Report the current state of every connected workstation so the monitoring page can refresh without reloading.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `TeacherController.LiveState (GET)`.
- Appears in the *MONITOR STUDENT SCREEN* module of the use case diagram.

Use Case Name: SEND SCREEN FRAME

Purpose: Send one captured frame of the workstation screen to the watching teacher.

Actors:

- Teacher (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: `frame : ScreenFrameMessage`

Output Parameters: A SignalR message delivered to the target workstation or group; no HTTP response.

Pre-Condition:

- The caller is signed in.
- **OPEN MONITORING** has reached the point where this is always performed.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Teacher triggers the command from the monitoring console.
2. The browser invokes `SendScreenFrame` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.SendScreenFrame (GET)`.
- Appears in the *MONITOR STUDENT SCREEN* module of the use case diagram.
- Drawn as <<include>> from **OPEN MONITORING**.

Control Student Workstation RemoteMonitoringHub

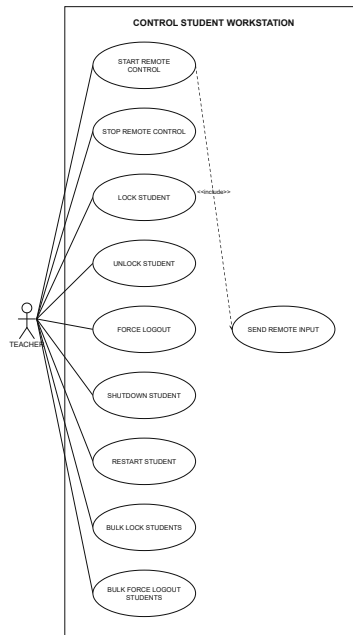


Figure 3.32: System Use Case for Control Student Workstation

Use Case Name: START REMOTE CONTROL

Purpose: Take keyboard and mouse control of a student workstation, with the client showing that remote control is active.

Actors:

- Teacher (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: `targetConnectionId: string`

Output Parameters: A `RemoteCommandResult` stating whether the workstation accepted the command, returned to the caller over SignalR.

Pre-Condition:

- The caller is signed in.
- The record named by the identifier exists.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Teacher triggers the command from the monitoring console.
2. The browser invokes `StartRemoteControl` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.
6. The command is written to the remote command log.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.StartRemoteControl (GET)`.
- Appears in the *CONTROL STUDENT WORKSTATION* module of the use case diagram.
- Pulls in **SEND REMOTE INPUT** (<<include>>).

Use Case Name: STOP REMOTE CONTROL

Purpose: Hand control of the workstation back to the student.

Actors:

- Teacher (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: `targetConnectionId: string`

Output Parameters: A `RemoteCommandResult` stating whether the workstation accepted the command, returned to the caller over SignalR.

Pre-Condition:

- The caller is signed in.
- The record named by the identifier exists.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Teacher triggers the command from the monitoring console.
2. The browser invokes `StopRemoteControl` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.
6. The command is written to the remote command log.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.StopRemoteControl (GET)`.
- Appears in the *CONTROL STUDENT WORKSTATION* module of the use case diagram.

Use Case Name: LOCK STUDENT

Purpose: Lock a student workstation so the student cannot use it until it is unlocked.

Actors:

- Teacher (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: `targetConnectionId: string`

Output Parameters: A SignalR message delivered to the target workstation or group; no HTTP response.

Pre-Condition:

- The caller is signed in.
- The record named by the identifier exists.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Teacher triggers the command from the monitoring console.
2. The browser invokes `LockStudent` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.
6. The command is written to the remote command log.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.LockStudent` (GET).
- Appears in the *CONTROL STUDENT WORKSTATION* module of the use case diagram.

Use Case Name: UNLOCK STUDENT

Purpose: Release the CAMS lock on a student workstation.

Actors:

- Teacher (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: `targetConnectionId: string`

Output Parameters: A SignalR message delivered to the target workstation or group; no HTTP response.

Pre-Condition:

- The caller is signed in.
- The record named by the identifier exists.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Teacher triggers the command from the monitoring console.
2. The browser invokes `UnlockStudent` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.UnlockStudent (GET)`.
- Appears in the *CONTROL STUDENT WORKSTATION* module of the use case diagram.

Use Case Name: FORCE LOGOUT

Purpose: Sign a student out of the workstation from the teacher console.

Actors:

- Teacher (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: `targetConnectionId: string`

Output Parameters: A SignalR message delivered to the target workstation or group; no HTTP response.

Pre-Condition:

- The caller is signed in.
- The record named by the identifier exists.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Teacher triggers the command from the monitoring console.
2. The browser invokes `ForceLogout` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.
6. The command is written to the remote command log.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.ForceLogout` (GET).
- Appears in the *CONTROL STUDENT WORKSTATION* module of the use case diagram.

Use Case Name: SHUTDOWN STUDENT

Purpose: Shut a student workstation down remotely.

Actors:

- Teacher (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: `targetConnectionId: string`

Output Parameters: A SignalR message delivered to the target workstation or group; no HTTP response.

Pre-Condition:

- The caller is signed in.
- The record named by the identifier exists.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Teacher triggers the command from the monitoring console.
2. The browser invokes `ShutdownStudent` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.
6. The command is written to the remote command log.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.ShutdownStudent` (GET).
- Appears in the *CONTROL STUDENT WORKSTATION* module of the use case diagram.

Use Case Name: RESTART STUDENT

Purpose: Restart a student workstation remotely.

Actors:

- Teacher (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: `targetConnectionId: string`

Output Parameters: A SignalR message delivered to the target workstation or group; no HTTP response.

Pre-Condition:

- The caller is signed in.
- The record named by the identifier exists.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Teacher triggers the command from the monitoring console.
2. The browser invokes `RestartStudent` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.
6. The command is written to the remote command log.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.RestartStudent (GET)`.
- Appears in the *CONTROL STUDENT WORKSTATION* module of the use case diagram.

Use Case Name: BULK LOCK STUDENTS

Purpose: Lock several student workstations in one action.

Actors:

- Teacher (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: `targetConnectionIds : List<string>`

Output Parameters: A SignalR message delivered to the target workstation or group; no HTTP response.

Pre-Condition: The caller is signed in.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Teacher triggers the command from the monitoring console.
2. The browser invokes `BulkLockStudents` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.
6. The command is written to the remote command log.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.BulkLockStudents (GET)`.

- Appears in the *CONTROL STUDENT WORKSTATION* module of the use case diagram.

Use Case Name: BULK FORCE LOGOUT STUDENTS

Purpose: Sign several students out of their workstations in one action.

Actors:

- Teacher (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: `targetConnectionIds : List<string>`

Output Parameters: A SignalR message delivered to the target workstation or group; no HTTP response.

Pre-Condition: The caller is signed in.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Teacher triggers the command from the monitoring console.
2. The browser invokes `BulkForceLogoutStudents` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.
6. The command is written to the remote command log.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.BulkForceLogoutStudents (GET)`.

- Appears in the *CONTROL STUDENT WORKSTATION* module of the use case diagram.

Use Case Name: SEND REMOTE INPUT

Purpose: Deliver one keyboard or mouse event to the workstation under remote control.

Actors:

- Teacher (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters:

- `targetConnectionId: string`
- `input: RemoteInputMessage`

Output Parameters: A `RemoteCommandResult` stating whether the workstation accepted the command, returned to the caller over SignalR.

Pre-Condition:

- The caller is signed in.
- The record named by the identifier exists.
- **START REMOTE CONTROL** has reached the point where this is always performed.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Teacher triggers the command from the monitoring console.
2. The browser invokes `SendRemoteInput` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.
6. The command is written to the remote command log.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.SendRemoteInput (GET)`.
- Appears in the *CONTROL STUDENT WORKSTATION* module of the use case diagram.
- Drawn as <<include>> from **START REMOTE CONTROL**.

Send Message To Student `RemoteMonitoringHub`

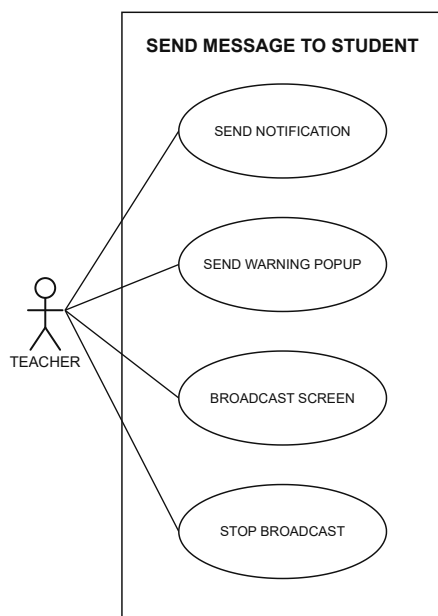


Figure 3.33: System Use Case for Send Message To Student

Use Case Name: SEND NOTIFICATION

Purpose: Send a message to connected students that appears on their workstation.

Actors:

- Teacher (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: `notification : NotificationMessage`

Output Parameters: A SignalR message delivered to the target workstation or group; no HTTP response.

Pre-Condition: The caller is signed in.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Teacher triggers the command from the monitoring console.
2. The browser invokes `SendNotification` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.SendNotification (GET)`.
- Appears in the *SEND MESSAGE TO STUDENT* module of the use case diagram.

Use Case Name: SEND WARNING POPUP

Purpose: Send a warning dialog to one student, shown on top of whatever they are doing.

Actors:

- Teacher (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters:

- `targetConnectionId:string`
- `warning:NotificationMessage`

Output Parameters: A SignalR message delivered to the target workstation or group; no HTTP response.

Pre-Condition:

- The caller is signed in.
- The record named by the identifier exists.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Teacher triggers the command from the monitoring console.
2. The browser invokes `SendWarningPopup` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **The identifier matches no record** — the action reports that the record was not found and makes no change.
- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.SendWarningPopup (GET)`.
- Appears in the *SEND MESSAGE TO STUDENT* module of the use case diagram.

Use Case Name: BROADCAST SCREEN

Purpose: Put the teacher screen on every connected student workstation.

Actors:

- Teacher (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: `frameBase64 : string`

Output Parameters: A SignalR message delivered to the target workstation or group; no HTTP response.

Pre-Condition: The caller is signed in.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Teacher triggers the command from the monitoring console.
2. The browser invokes `BroadcastScreen` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.BroadcastScreen (GET)`.
- Appears in the *SEND MESSAGE TO STUDENT* module of the use case diagram.

Use Case Name: STOP BROADCAST

Purpose: Stop the teacher screen broadcast and return the workstations to the student view.

Actors:

- Teacher (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: A SignalR message delivered to the target workstation or group; no HTTP response.

Pre-Condition: The caller is signed in.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Teacher triggers the command from the monitoring console.
2. The browser invokes `StopBroadcast` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.StopBroadcast (GET)`.
- Appears in the *SEND MESSAGE TO STUDENT* module of the use case diagram.

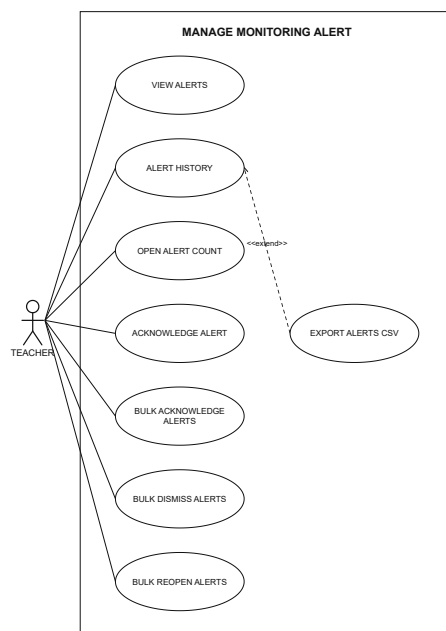
Manage Monitoring Alert TeacherController

Figure 3.34: System Use Case for Manage Monitoring Alert

Use Case Name: VIEW ALERTS

Purpose: List the monitoring alerts raised for the classes the teacher is responsible for.

Actors:

- Teacher (Primary Actor)

Input Parameters: `filter : AlertListFilter`

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `TeacherController.Alerts (GET)`.
- Appears in the *MANAGE MONITORING ALERT* module of the use case diagram.

Use Case Name: ALERT HISTORY

Purpose: Show alerts that have already been acted on, with who acted and when.

Actors:

- Teacher (Primary Actor)

Input Parameters: `id:int`

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.AlertHistory` (GET).
- Appears in the *MANAGE MONITORING ALERT* module of the use case diagram.
- Pulls in **EXPORT ALERTS CSV** (<<extend>>).

Use Case Name: OPEN ALERT COUNT

Purpose: Report how many alerts are still open, for the badge on the navigation bar.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `TeacherController.OpenAlertCount (GET)`.
- Appears in the *MANAGE MONITORING ALERT* module of the use case diagram.

Use Case Name: ACKNOWLEDGE ALERT

Purpose: Mark one alert as seen and being handled.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `id:int`
- `acknowledged:bool`
- `filter:AlertListFilter`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.AcknowledgeAlert` (POST).
- Appears in the *MANAGE MONITORING ALERT* module of the use case diagram.

Use Case Name: BULK ACKNOWLEDGE ALERTS

Purpose: Acknowledge several alerts in one action.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `alertIds : List<int>?`
- `filter : AlertListFilter`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `TeacherController.BulkAcknowledgeAlerts` (POST).
- Appears in the *MANAGE MONITORING ALERT* module of the use case diagram.

Use Case Name: BULK DISMISS ALERTS

Purpose: Dismiss several alerts in one action.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `alertIds : List<int>?`
- `reason : string?`
- `filter : AlertListFilter`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `TeacherController.BulkDismissAlerts` (POST).
- Appears in the *MANAGE MONITORING ALERT* module of the use case diagram.

Use Case Name: BULK REOPEN ALERTS

Purpose: Reopen several alerts that were closed too early.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `alertIds : List<int>?`
- `filter : AlertListFilter`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The Teacher fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `TeacherController.BulkReopenAlerts` (POST).
- Appears in the *MANAGE MONITORING ALERT* module of the use case diagram.

Use Case Name: EXPORT ALERTS CSV

Purpose: Produce the alerts view as a CSV file the user can download.

Actors:

- Teacher (Primary Actor)
- None; this behaviour runs inside **ALERT HISTORY** (Secondary Actor)

Input Parameters: `filter : AlertListFilter`

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as a teacher.
- **ALERT HISTORY** has reached the point where this is optionally performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `TeacherController.ExportAlertsCsv` (GET).
- Appears in the *MANAGE MONITORING ALERT* module of the use case diagram.
- Drawn as <<extend>> to **ALERT HISTORY**.

View Teacher Records TeacherController

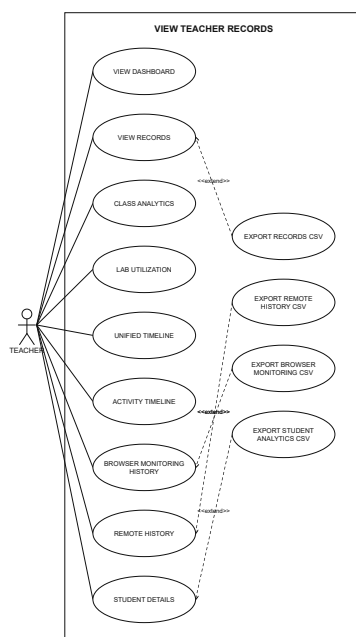


Figure 3.35: System Use Case for View Teacher Records

Use Case Name: VIEW DASHBOARD

Purpose: Show the teacher landing page with the state of the laboratory at a glance.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `TeacherController.Dashboard (GET)`.
- Appears in the *VIEW TEACHER RECORDS* module of the use case diagram.

Use Case Name: VIEW RECORDS

Purpose: Show captured application and website activity for the teacher's students.

Actors:

- Teacher (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `TeacherController.Records (GET)`.
- Appears in the *VIEW TEACHER RECORDS* module of the use case diagram.
- Pulls in **EXPORT RECORDS CSV** (<<extend>>).

Use Case Name: CLASS ANALYTICS

Purpose: Show usage patterns for a class rather than for one student.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `id:int`
- `from:DateTime?` (optional)
- `to:DateTime?` (optional)
- `station:string?` (optional)

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.ClassAnalytics` (GET).
- Appears in the *VIEW TEACHER RECORDS* module of the use case diagram.

Use Case Name: LAB UTILIZATION

Purpose: Show how heavily the laboratory workstations are being used over time.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `from:DateTime?` (optional)
- `to:DateTime?` (optional)
- `station:string?` (optional)
- `classId:int?` (optional)

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.LabUtilization` (GET).
- Appears in the *VIEW TEACHER RECORDS* module of the use case diagram.

Use Case Name: UNIFIED TIMELINE

Purpose: Show one student's activity as a single timeline across applications, websites and idle periods.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `from:DateTime?` (optional)
- `to:DateTime?` (optional)
- `studentId:int?` (optional)
- `classId:int?` (optional)
- `station:string?` (optional)
- `source:string?` (optional)
- `eventType:string?` (optional)
- `page:int` (optional)
- `pageSize:int` (optional)

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.

3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.UnifiedTimeline (GET)`.
- Appears in the *VIEW TEACHER RECORDS* module of the use case diagram.

Use Case Name: ACTIVITY TIMELINE

Purpose: Serve the timeline data the unified timeline view is drawn from.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `id: int`
- `from: DateTime?` (optional)
- `to: DateTime?` (optional)
- `page: int` (optional)
- `pageSize: int` (optional)
- `eventType: string?` (optional)

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.ActivityTimeline` (GET).
- Appears in the *VIEW TEACHER RECORDS* module of the use case diagram.

Use Case Name: BROWSER MONITORING HISTORY

Purpose: Show the record of browser activity captured from the workstations.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `from : DateTime?` (optional)
- `to : DateTime?` (optional)
- `browser : string?` (optional)
- `mode : string?` (optional)
- `page : int` (optional)
- `pageSize : int` (optional)

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as a teacher.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `TeacherController.BrowserMonitoringHistory (GET)`.

- Appears in the *VIEW TEACHER RECORDS* module of the use case diagram.
- Pulls in **EXPORT BROWSER MONITORING CSV** (<<extend>>).

Use Case Name: REMOTE HISTORY

Purpose: Show which remote commands were issued, by whom, and against which workstation.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `from:DateTime?` (optional)
- `to:DateTime?` (optional)
- `command:string?` (optional)
- `studentId:string?` (optional)
- `page:int` (optional)

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.RemoteHistory` (GET).
- Appears in the *VIEW TEACHER RECORDS* module of the use case diagram.
- Pulls in **EXPORT REMOTE HISTORY CSV** (<<extend>>).

Use Case Name: STUDENT DETAILS

Purpose: Show one student in full: account, class, sessions and captured activity.

Actors:

- Teacher (Primary Actor)

Input Parameters:

- `id:int`
- `from:DateTime?` (optional)
- `to:DateTime?` (optional)

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.StudentDetails` (GET).
- Appears in the *VIEW TEACHER RECORDS* module of the use case diagram.
- Pulls in **EXPORT STUDENT ANALYTICS CSV** (<<extend>>).

Use Case Name: EXPORT RECORDS CSV

Purpose: Produce the records view as a CSV file the user can download.

Actors:

- Teacher (Primary Actor)
- None; this behaviour runs inside **VIEW RECORDS** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as a teacher.
- **VIEW RECORDS** has reached the point where this is optionally performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `TeacherController.ExportRecordsCsv` (GET).
- Appears in the *VIEW TEACHER RECORDS* module of the use case diagram.
- Drawn as <<extend>> to **VIEW RECORDS**.

Use Case Name: EXPORT REMOTE HISTORY CSV

Purpose: Produce the remote history view as a CSV file the user can download.

Actors:

- Teacher (Primary Actor)
- None; this behaviour runs inside **REMOTE HISTORY** (Secondary Actor)

Input Parameters:

- `from: DateTime?` (optional)
- `to: DateTime?` (optional)
- `command: string?` (optional)
- `studentId: string?` (optional)

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.
- **REMOTE HISTORY** has reached the point where this is optionally performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.ExportRemoteHistoryCsv` (GET).
- Appears in the *VIEW TEACHER RECORDS* module of the use case diagram.
- Drawn as <<extend>> to **REMOTE HISTORY**.

Use Case Name: EXPORT BROWSER MONITORING CSV

Purpose: Produce the browser monitoring view as a CSV file the user can download.

Actors:

- Teacher (Primary Actor)
- None; this behaviour runs inside **BROWSER MONITORING HISTORY** (Secondary Actor)

Input Parameters:

- `from:DateTime?` (optional)
- `to:DateTime?` (optional)
- `browser:string?` (optional)
- `mode:string?` (optional)

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as a teacher.
- **BROWSER MONITORING HISTORY** has reached the point where this is optionally performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `TeacherController.ExportBrowserMonitoringCsv` (GET).
- Appears in the *VIEW TEACHER RECORDS* module of the use case diagram.
- Drawn as <<extend>> to **BROWSER MONITORING HISTORY**.

Use Case Name: EXPORT STUDENT ANALYTICS CSV

Purpose: Produce the student analytics view as a CSV file the user can download.

Actors:

- Teacher (Primary Actor)
- None; this behaviour runs inside **STUDENT DETAILS** (Secondary Actor)

Input Parameters:

- `id:int`
- `from:DateTime?` (optional)
- `to:DateTime?` (optional)

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition:

- The caller is signed in as a teacher.
- The record named by the identifier exists.
- **STUDENT DETAILS** has reached the point where this is optionally performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Teacher opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `TeacherController.ExportStudentAnalyticsCsv (GET)`.
- Appears in the *VIEW TEACHER RECORDS* module of the use case diagram.
- Drawn as <<extend>> to **STUDENT DETAILS**.

STUDENT

Process Log In AccountController

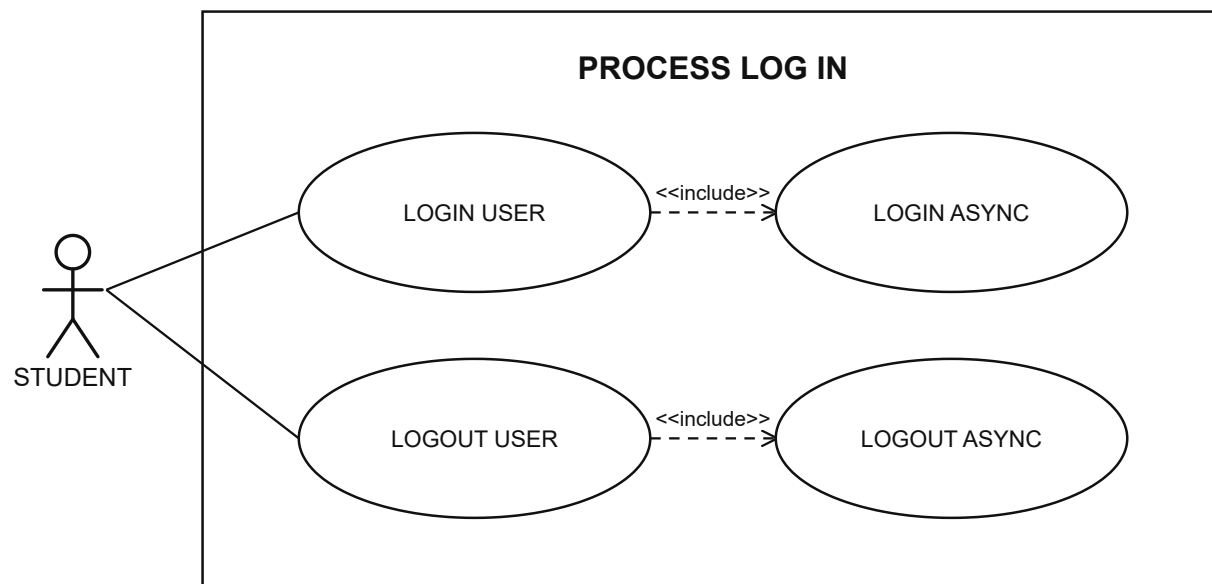


Figure 3.36: System Use Case for Process Log In

Use Case Name: LOGIN USER

Purpose: Let a person sign in to the CAMS web portal with a username and password, and place them in the part of the system their role allows.

Actors:

- Student (Primary Actor)

Input Parameters:

- `username : string`
- `password : string`

Output Parameters: An authentication cookie carrying the account role as a claim, and a redirect to the landing page for that role.

Pre-Condition: None. This is the sign-in endpoint and is reachable without an account session; the CAMS server must be running and reachable.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Student opens the CAMS sign-in page.
2. The Student enters a username and a password and submits the form.
3. The server validates the antiforgery token that accompanied the form.
4. The server checks the credentials against each account table in turn and finds the matching account.
5. The server confirms the account is active and not locked out.
6. The server issues an authentication cookie carrying the role as a claim.
7. The Student is redirected to the landing page for that role.

Exception Scenario:

- **The username matches no account** — the page reports that the sign-in failed, without saying which half was wrong.
- **The password does not match the stored hash** — the failed-attempt counter is raised and the same message is shown.
- **The account is locked out** — the sign-in is refused until the lockout expires, even with the right password.
- **The account is deactivated** — the sign-in is refused and the person is told to contact an administrator.

Additional Remarks:

- Implemented by `AccountController.Login` (POST).
- Appears in the *PROCESS LOG IN* module of the use case diagram.
- Pulls in **LOGIN ASYNC** (<<include>>).

Use Case Name: LOGOUT USER

Purpose: End the signed-in session and clear the authentication cookie, so the next visitor to the browser starts as an anonymous user.

Actors:

- Student (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The authentication cookie is cleared and the browser is returned to the sign-in page.

Pre-Condition: The caller is signed in.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Student chooses to sign out.
2. The server closes any lab session the account still has open.
3. The server clears the authentication cookie.
4. The browser is returned to the sign-in page as an anonymous visitor.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `AccountController.Logout` (POST).
- Appears in the *PROCESS LOG IN* module of the use case diagram.

- Pulls in **LOGOUT ASYNC** (<<include>>).

Use Case Name: LOGIN ASYNC

Purpose: Check the supplied password against the stored hash, confirm the account is active and not locked out, and decide which of the three account tables the person belongs to.

Actors:

- Student (Primary Actor)
- None; this behaviour runs inside **LOGIN USER** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: A `LoginResult` naming the role, the account id and the display name, or a result saying the credentials were rejected.

Pre-Condition:

- The sign-in use case has supplied a username and a password.
- **LOGIN USER** has reached the point where this is always performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The sign-in use case passes the username, the password, the workstation name and the caller address.
2. The service looks the username up in the administrator, teacher and student tables.
3. The stored hash is verified against the supplied password.
4. The active flag and the lockout expiry are checked.
5. On success the failed-attempt counter is cleared and the role and account id are returned; on failure the counter is raised.

Exception Scenario:

- The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.

Additional Remarks:

- Implemented by `LoginAsync`.
- Appears in the *PROCESS LOG IN* module of the use case diagram.
- Drawn as `<<include>>` from **LOGIN USER**.

Use Case Name: LOGOUT ASYNC

Purpose: Close any lab session the student still has open and record the sign-out.

Actors:

- Student (Primary Actor)
- None; this behaviour runs inside **LOGOUT USER** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The behaviour completes and its effect is visible to the use case that includes it.

Pre-Condition:

- The caller is signed in.
- **LOGOUT USER** has reached the point where this is always performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The including use case reaches the point where this behaviour is required.
2. The server runs `LogoutAsync` and applies its result.
3. Control returns to the including use case, which continues.

Exception Scenario:

- The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.

Additional Remarks:

- Implemented by `LogoutAsync`.
- Appears in the *PROCESS LOG IN* module of the use case diagram.
- Drawn as <<include>> from **LOGOUT USER**.

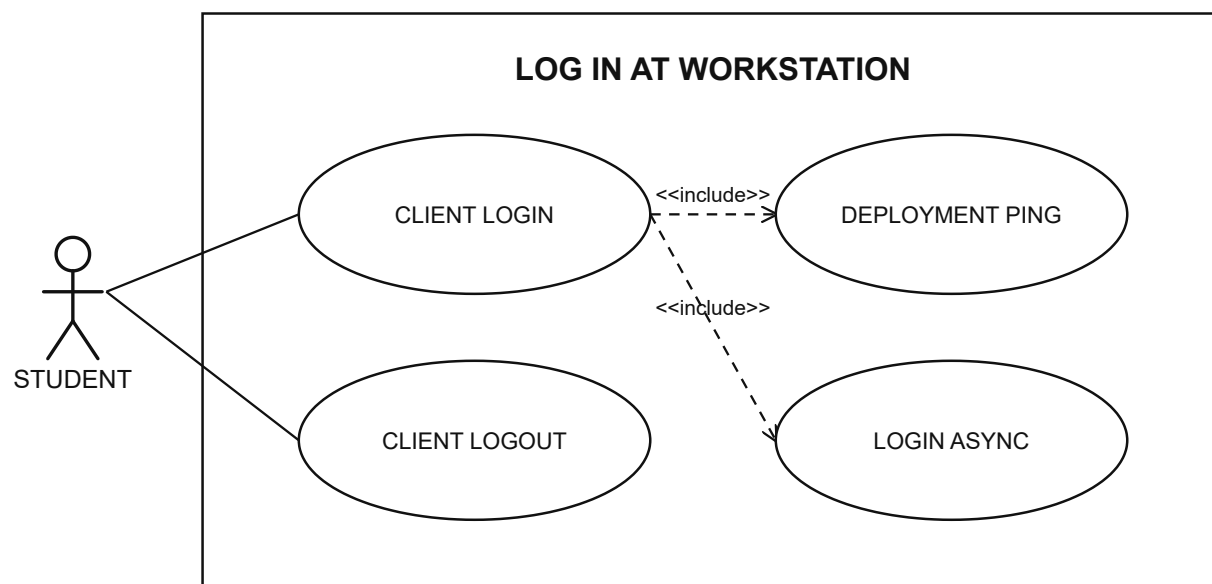
Log In At Workstation `ClientAuthController`

Figure 3.37: System Use Case for Log In At Workstation

Use Case Name: CLIENT LOGIN

Purpose: Let a student sign in from the CAMS client installed on a laboratory workstation, binding the sign-in to the machine the student is sitting at.

Actors:

- Student (Primary Actor)

Input Parameters: `request : StudentClientLoginRequest`

Output Parameters: A `StudentClientLoginResponse` carrying the student identity, the workstation registration and the session state.

Pre-Condition: None. The client reaches this before any session exists. The workstation must have found the server and must trust the CAMS root certificate.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The CAMS client finds the server on the laboratory network.
2. The student enters a username and a password in the client.
3. The client posts the credentials together with the name of the workstation.
4. The server verifies the credentials and confirms the account is a student that is active and not locked out.
5. The server registers the workstation against the student, reassigning it if it was held by someone else who is no longer using it.
6. The client receives the session details and the monitored session begins.

Exception Scenario:

- **No server was found on the network** — the client reports that it cannot reach CAMS and offers the endpoint to be entered by hand.

- **The workstation is already held by another active session** — the sign-in is refused rather than displacing the student using it.
- **The account is not a student** — the client refuses the sign-in; teacher and administrator accounts use the web portal.

Additional Remarks:

- Implemented by `ClientAuthController.Login` (POST).
- Appears in the *LOG IN AT WORKSTATION* module of the use case diagram.
- Pulls in **DEPLOYMENT PING** (<<include>>), **LOGIN ASYNC** (<<include>>).

Use Case Name: CLIENT LOGOUT

Purpose: End the workstation session from the client, releasing the workstation so another student may sign in to it.

Actors:

- Student (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: Confirmation that the session was closed and the workstation released.

Pre-Condition: The caller is signed in as a student.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The student signs out from the CAMS client.
2. The server ends the lab session and records its end time.
3. The workstation is released so another student may sign in at it.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `ClientAuthController.Logout` (POST).
- Appears in the *LOG IN AT WORKSTATION* module of the use case diagram.

Use Case Name: DEPLOYMENT PING

Purpose: Answer the discovery request a client broadcasts while looking for the CAMS server on the laboratory network.

Actors:

- Student (Primary Actor)
- None; this behaviour runs inside **CLIENT LOGIN** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The server endpoint a client should connect to.

Pre-Condition:

- None. The endpoint answers an unauthenticated broadcast, so a client can find the server before it has credentials.
- **CLIENT LOGIN** has reached the point where this is always performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. A CAMS client broadcasts on the laboratory network looking for a server.
2. The server answers with the endpoint the client should use.
3. The client stores the endpoint and proceeds to sign in.

Exception Scenario:

- **The broadcast does not reach the server** — no answer is returned and the client falls back to a configured endpoint.

Additional Remarks:

- Implemented by `DeploymentPingController.Get (GET)`.
- Appears in the *LOG IN AT WORKSTATION* module of the use case diagram.
- Drawn as <<include>> from **CLIENT LOGIN**.

Use Case Name: LOGIN ASYNC

Purpose: Check the supplied password against the stored hash, confirm the account is active and not locked out, and decide which of the three account tables the person belongs to.

Actors:

- Student (Primary Actor)
- None; this behaviour runs inside **CLIENT LOGIN** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: A `LoginResult` naming the role, the account id and the display name, or a result saying the credentials were rejected.

Pre-Condition:

- The sign-in use case has supplied a username and a password.
- **CLIENT LOGIN** has reached the point where this is always performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The sign-in use case passes the username, the password, the workstation name and the caller address.
2. The service looks the username up in the administrator, teacher and student tables.
3. The stored hash is verified against the supplied password.
4. The active flag and the lockout expiry are checked.
5. On success the failed-attempt counter is cleared and the role and account id are returned; on failure the counter is raised.

Exception Scenario:

- The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.

Additional Remarks:

- Implemented by `LoginAsync`.
- Appears in the *LOG IN AT WORKSTATION* module of the use case diagram.
- Drawn as `<<include>>` from **CLIENT LOGIN**.

Work At Monitored Workstation RemoteMonitoringHub

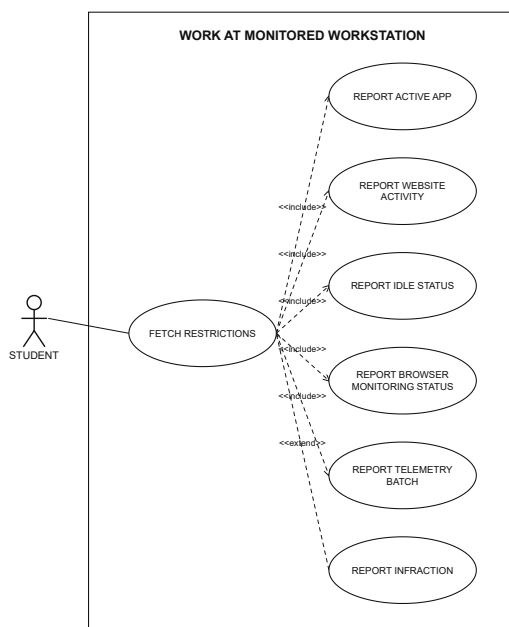


Figure 3.38: System Use Case for Work At Monitored Workstation

Use Case Name: FETCH RESTRICTIONS

Purpose: Give the client the restriction rules that apply to the student signed in at that workstation.

Actors:

- Student (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: A SignalR message delivered to the target workstation or group; no HTTP response.

Pre-Condition: The caller is signed in.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Student triggers the command from the monitoring console.
2. The browser invokes `FetchRestrictions` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.FetchRestrictions (GET)`.

- Appears in the *WORK AT MONITORED WORKSTATION* module of the use case diagram.
- Pulls in **REPORT ACTIVE APP** (<<include>>), **REPORT WEBSITE ACTIVITY** (<<include>>), **REPORT IDLE STATUS** (<<include>>), **REPORT BROWSER MONITORING STATUS** (<<include>>), **REPORT TELEMETRY BATCH** (<<include>>), **REPORT INFRACTION** (<<extend>>).

Use Case Name: REPORT ACTIVE APP

Purpose: Report which application is in the foreground on the workstation.

Actors:

- Student (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: `app : ActiveAppMessage`

Output Parameters: A SignalR message delivered to the target workstation or group; no HTTP response.

Pre-Condition:

- The caller is signed in.
- **FETCH RESTRICTIONS** has reached the point where this is always performed.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Student triggers the command from the monitoring console.
2. The browser invokes `ReportActiveApp` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.ReportActiveApp` (GET).
- Appears in the *WORK AT MONITORED WORKSTATION* module of the use case diagram.
- Drawn as `<<include>>` from **FETCH RESTRICTIONS**.

Use Case Name: REPORT WEBSITE ACTIVITY

Purpose: Report the website the student is viewing in the browser.

Actors:

- Student (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: `website : WebsiteActivityMessage`

Output Parameters: A SignalR message delivered to the target workstation or group; no HTTP response.

Pre-Condition:

- The caller is signed in.
- **FETCH RESTRICTIONS** has reached the point where this is always performed.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Student triggers the command from the monitoring console.
2. The browser invokes `ReportWebsiteActivity` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.ReportWebsiteActivity (GET)`.
- Appears in the *WORK AT MONITORED WORKSTATION* module of the use case diagram.
- Drawn as `<<include>>` from **FETCH RESTRICTIONS**.

Use Case Name: REPORT IDLE STATUS

Purpose: Report whether the workstation has gone idle.

Actors:

- Student (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: `status : IdleStatusMessage`

Output Parameters: A SignalR message delivered to the target workstation or group; no HTTP response.

Pre-Condition:

- The caller is signed in.
- **FETCH RESTRICTIONS** has reached the point where this is always performed.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Student triggers the command from the monitoring console.
2. The browser invokes `ReportIdleStatus` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.ReportIdleStatus` (GET).
- Appears in the *WORK AT MONITORED WORKSTATION* module of the use case diagram.
- Drawn as `<<include>>` from **FETCH RESTRICTIONS**.

Use Case Name: REPORT BROWSER MONITORING STATUS

Purpose: Report whether browser monitoring is working on the workstation.

Actors:

- Student (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: `status : BrowserMonitoringStatusMessage`

Output Parameters: A SignalR message delivered to the target workstation or group; no HTTP response.

Pre-Condition:

- The caller is signed in.
- **FETCH RESTRICTIONS** has reached the point where this is always performed.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Student triggers the command from the monitoring console.
2. The browser invokes `ReportBrowserMonitoringStatus` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.ReportBrowserMonitoringStatus` (GET).
- Appears in the *WORK AT MONITORED WORKSTATION* module of the use case diagram.
- Drawn as <<include>> from **FETCH RESTRICTIONS**.

Use Case Name: REPORT TELEMETRY BATCH

Purpose: Send a batch of buffered telemetry, so a brief disconnection does not lose the record.

Actors:

- Student (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: `batch : TelemetryBatchMessage`

Output Parameters: A SignalR message delivered to the target workstation or group; no HTTP response.

Pre-Condition:

- The caller is signed in.
- **FETCH RESTRICTIONS** has reached the point where this is always performed.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Student triggers the command from the monitoring console.
2. The browser invokes `ReportTelemetryBatch` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.ReportTelemetryBatch (GET)`.
- Appears in the *WORK AT MONITORED WORKSTATION* module of the use case diagram.
- Drawn as `<<include>>` from **FETCH RESTRICTIONS**.

Use Case Name: REPORT INFRACTION

Purpose: Report that the student tried to open something a restriction rule blocks.

Actors:

- Student (Primary Actor)
- The CAMS client on the target workstation (Secondary Actor)

Input Parameters: `infraction: InfractionMessage`

Output Parameters: A SignalR message delivered to the target workstation or group; no HTTP response.

Pre-Condition:

- The caller is signed in.
- **FETCH RESTRICTIONS** has reached the point where this is optionally performed.

Post-Condition: The workstation has acted on the command and the console shows its new state.

Successful Scenario:

1. The Student triggers the command from the monitoring console.
2. The browser invokes `ReportInfraction` on the SignalR hub over the open connection.
3. The server checks the caller's role and resolves the target connection.
4. The server relays the instruction to the workstation client.
5. The client carries it out and the console reflects the new state.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **The workstation is not connected** — the command cannot be delivered and the console reports the workstation as offline.

Additional Remarks:

- Implemented by `RemoteMonitoringHub.ReportInfraction (GET)`.
- Appears in the *WORK AT MONITORED WORKSTATION* module of the use case diagram.
- Drawn as <<extend>> to **FETCH RESTRICTIONS**.

View Student Session `StudentController`

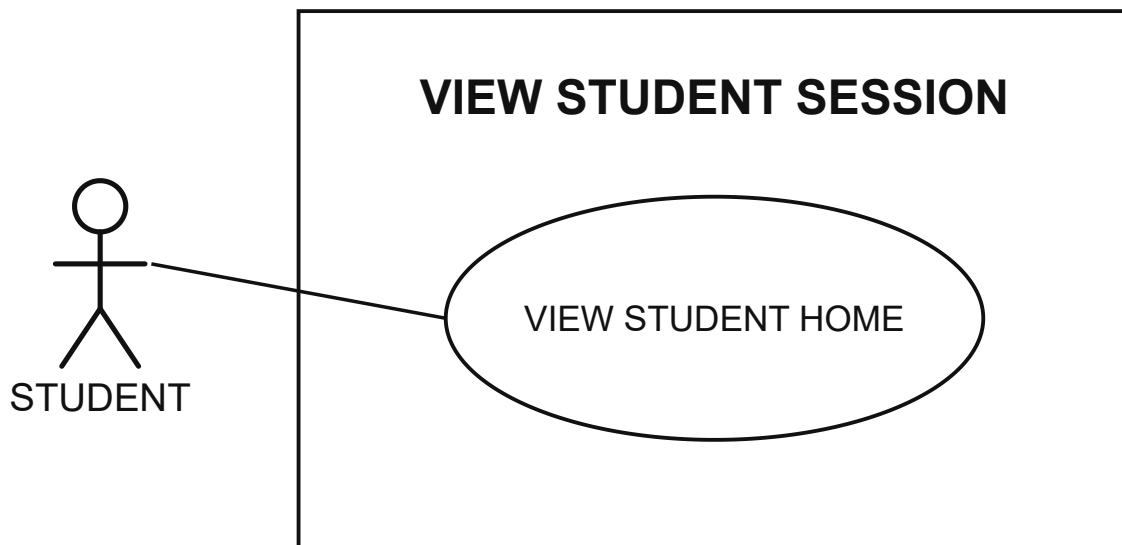


Figure 3.39: System Use Case for View Student Session

Use Case Name: VIEW STUDENT HOME

Purpose: Show the student portal home with the current session state and timer.

Actors:

- Student (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as a student.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Student opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `StudentController.Index` (GET).
- Appears in the *VIEW STUDENT SESSION* module of the use case diagram.

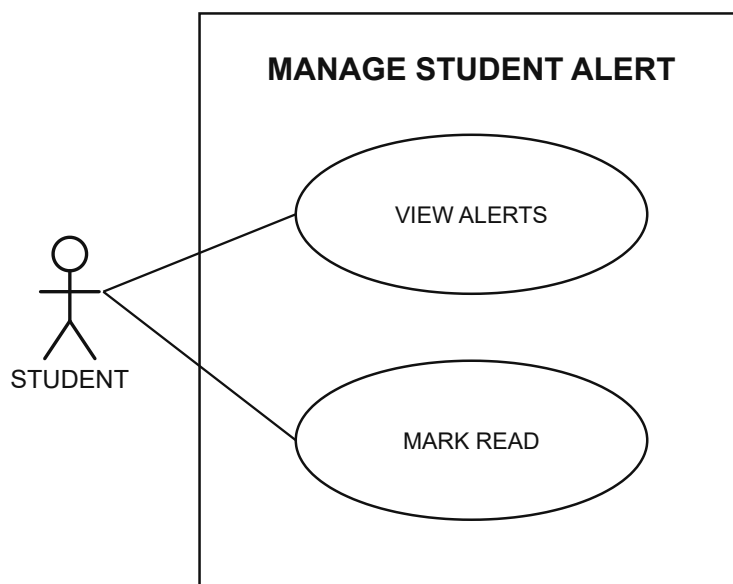
Manage Student Alert StudentController

Figure 3.40: System Use Case for Manage Student Alert

Use Case Name: VIEW ALERTS

Purpose: List the monitoring alerts raised for the classes the teacher is responsible for.

Actors:

- Student (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as a student.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Student opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `StudentController.Alerts (GET)`.
- Appears in the *MANAGE STUDENT ALERT* module of the use case diagram.

Use Case Name: MARK READ

Purpose: Let a student mark one of their own alerts as read.

Actors:

- Student (Primary Actor)

Input Parameters: `id: int`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition:

- The caller is signed in as a student.
- The record named by the identifier exists.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Student opens the page and CAMS confirms the role on the authentication cookie.
2. The Student fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.
- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.
- **The identifier matches no record** — the action reports that the record was not found and makes no change.

Additional Remarks:

- Implemented by `StudentController.MarkRead (POST)`.
- Appears in the *MANAGE STUDENT ALERT* module of the use case diagram.

Manage Student Account `StudentController`

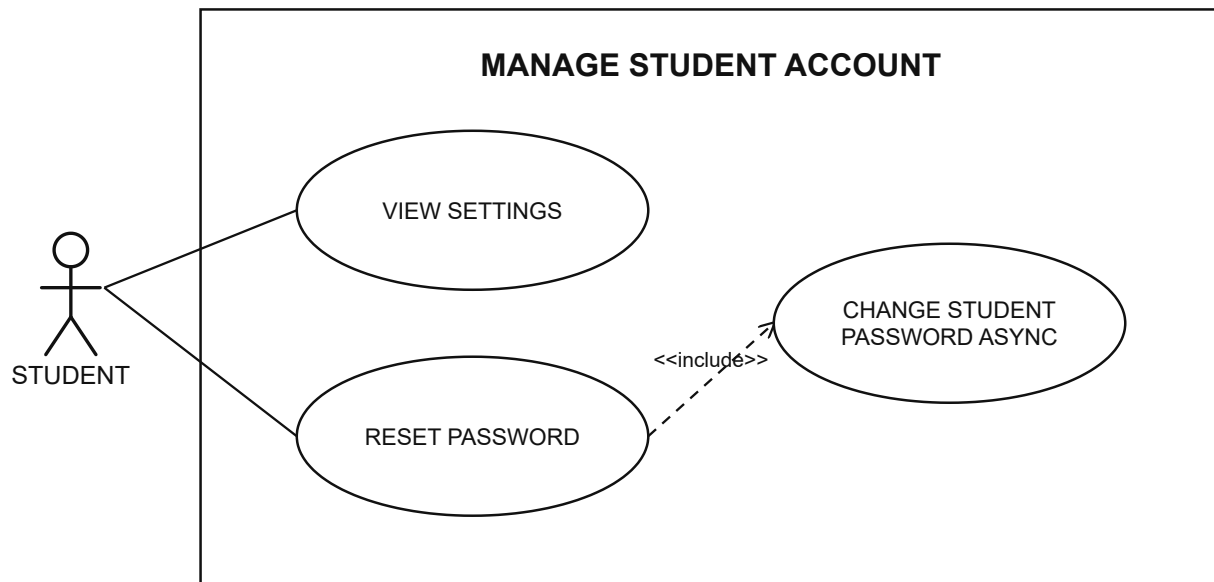


Figure 3.41: System Use Case for Manage Student Account

Use Case Name: VIEW SETTINGS

Purpose: Show the settings page for the signed-in user.

Actors:

- Student (Primary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The rendered page, or a JSON payload where the caller is the page's own script.

Pre-Condition: The caller is signed in as a student.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The Student opens the page and CAMS confirms the role on the authentication cookie.
2. The server reads the records the caller is entitled to see.
3. The page renders with those records.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.

Additional Remarks:

- Implemented by `StudentController.Settings (GET)`.
- Appears in the *MANAGE STUDENT ACCOUNT* module of the use case diagram.

Use Case Name: RESET PASSWORD

Purpose: Let a student replace their own password from the student portal.

Actors:

- Student (Primary Actor)

Input Parameters: `input : PasswordChangeInput`

Output Parameters: A redirect back to the listing page, carrying a success or failure message for display.

Pre-Condition: The caller is signed in as a student.

Post-Condition:

- The change is committed to the database and visible to the next read.
- The caller sees the outcome reported on the page they return to.

Successful Scenario:

1. The Student opens the page and CAMS confirms the role on the authentication cookie.
2. The Student fills the form and submits it.
3. The server validates the antiforgery token that accompanied the form.
4. The server validates the submitted values against the model rules.
5. The change is written to the database through `ApplicationDbContext`.
6. The server redirects back to the listing, where the result is shown.

Exception Scenario:

- **Not signed in or wrong role** — the request is refused and the caller is sent to the access denied page.
- **Missing or stale antiforgery token** — the submission is rejected and must be retried from a freshly loaded form.

- **Validation fails** — the form is redisplayed with the offending fields marked and nothing is written.

Additional Remarks:

- Implemented by `StudentController.ResetPassword` (POST).
- Appears in the *MANAGE STUDENT ACCOUNT* module of the use case diagram.
- Pulls in **CHANGE STUDENT PASSWORD ASYNC** (<<include>>).

Use Case Name: CHANGE STUDENT PASSWORD ASYNC

Purpose: Verify the current password and replace the stored hash for a student account.

Actors:

- Student (Primary Actor)
- None; this behaviour runs inside **RESET PASSWORD** (Secondary Actor)

Input Parameters: None beyond the signed-in identity carried on the authentication cookie.

Output Parameters: The behaviour completes and its effect is visible to the use case that includes it.

Pre-Condition:

- The caller is signed in.
- **RESET PASSWORD** has reached the point where this is always performed.

Post-Condition: The caller has the requested information. Nothing in the database has changed.

Successful Scenario:

1. The including use case reaches the point where this behaviour is required.
2. The server runs `ChangeStudentPasswordAsync` and applies its result.
3. Control returns to the including use case, which continues.

Exception Scenario:

- The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.

Additional Remarks:

- Implemented by `ChangeStudentPasswordAsync`.
- Appears in the *MANAGE STUDENT ACCOUNT* module of the use case diagram.
- Drawn as <<include>> from **RESET PASSWORD**.